RC522双端Zigbee考勤系统

代码阅读与答辩手册

阅读对象为当前普通版 RC522大作业。源码核查日期为2026年10月13日。先掌握两端职责和调度，再沿真实业务追到通信、数据库和驱动。阅读结束后，应能回答每个功能从哪里触发、由谁执行、数据写在哪里、什么时候才算成功、失败后如何处理。

本系统采用CC2530和Z-Stack 2.5.1a。协调器负责建网、统一时间、远程管理和本机读卡；终端负责唯一的名单及考勤数据库。两端都能读卡，但两端共用终端AT24C08中的8个用户槽和10条循环考勤。协调器AT24C02保存本机统计，不保存第二份人员名单。当前普通版禁用ESP12F；另一份ESP12F MQTT项目和网站是扩展版本。

阅读目录

如何使用这份手册 3

系统全貌与职责 3

分阶段阅读顺序 5

启动过程和自动组网 6

OSAL消息和事件调度 7

外设初始化及关键状态 8

按键 注册 注销和退出 9

两条考勤链路 10

无线协议和事务可靠性 11

AT24C08数据库和掉电恢复 13

EEPROM驱动和协调器本机统计 15

RTC统一时间与有效性 16

串口命令与非阻塞输出 17

OLED 中文页面与增量刷新 19

底层器件逐个阅读 19

返回值 异常和资源约束 20

答辩演示和验证依据 21

答辩问题和回答依据 23

扩展版本和后续修改位置 30

全部文件和函数阅读索引 31

源码和核查资料 52

如何使用这份手册

阅读时同时打开IAR和本手册。章节中的“源码定位”给出当前函数起始行；阅读版链接可以跳转文件。附录列出全部204个自写C函数，并标记其调用关系和用途。SDK内部函数只展开与启动、调度、组网、收发和板级适配直接相关的路径；协议栈二进制库不能逐行阅读。

建议完成三轮阅读。第一轮读系统全貌、阅读路线、OSAL和两条刷卡链路，先讲得出完整流程。第二轮读协议、数据库、RTC、串口、OLED和驱动，把字段、状态机和异常处理解释清楚。第三轮按答辩问题反查函数，再用演示步骤验证自己能讲清代码。

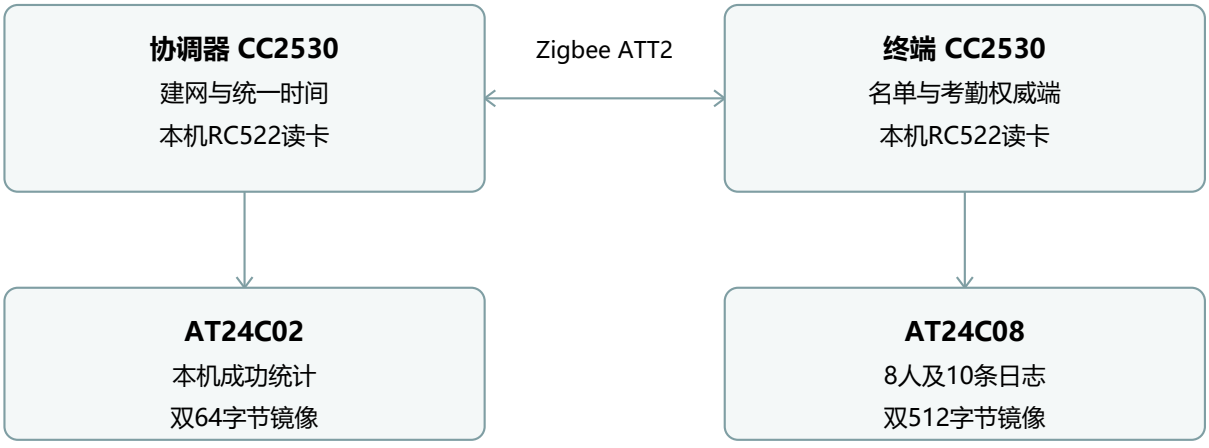
参考项目dazuoye用于理解原SampleApp结构；Legacy、docs/backups、Obj、List、Exe及外设参考代码不等于当前在用业务。不要从历史文件推断当前串口命令或驱动结构。

系统全貌与职责

两端各自做什么

项目	协调器	终端
网络角色	自动建立网络，短地址通常为0000	自动加入网络，通过父节点取得下行消息
RFID读卡	读取本机UID，发ATT_SCAN请求	读取本机UID，直接执行本地数据库操作
名单和考勤	向终端查询和修改	AT24C08为唯一权威数据库
时钟	RX8010作为统一时间源	先向协调器取时，再由本机RX8010走时
本机存储	AT24C02保存启动次数和本机成功操作统计	AT24C08保存人员、日志和重放保护字段
离线刷卡	无法确认远程落库，没有离线补传队列	可以本地落库；无有效时间则标记valid为0
注册注销	按键或串口发送模式请求	维护权威模式及名单，完成持久化后应答

本项目是“一台协调器加一台业务终端”的实现。协调器只维护一个终端地址，在线期间会拒绝另一个短地址抢占。8人指8张注册卡的数据库容量，不是8台终端。



分层及文件夹

Source中的Coordinator.c和EndDevice.c负责应用任务组装、消息分发及定时事件调度。app负责考勤、注册、注销、协议动作、数据库和页面选择。hal负责引脚、I2C、SPI、UART、RTC和显示器件的具体操作。app/Common保留旧项目的串口、单播、广播和协议阅读方式。

SDK的Components/hal与项目自己的hal并存。LED直接调用SDK的HalLedSet；按键GPIO采样调用SDK的HalKeyRead，项目再做消抖。蜂鸣器单独在hal_buzzer.c中。引脚入口统一为hal/board/board_config.h，SDK hal_board_cfg.h通过GAOCHANG_RC522_BOARD条件适配这块自制板。

源码定位：SampleApp_Init 第47行；SampleApp_Init 第46行；hal/board/board_config.h。

从集中配置查全部引脚

器件或信号	CC2530引脚	代码中的方式
LED	P1.0	高电平亮，SDK驱动
蜂鸣器	P0.0	高电平响，独立驱动
按键1及按键2	P0.1及P2.0	低电平按下，SDK采样后项目消抖
I2C SCL及SDA	P1.2及P1.1	输出0或切输入释放，外部上拉
RC522 CS及CLK	P1.4及P1.5	软件SPI选中及移位时钟
RC522 MOSI及MISO	P1.6及P1.7	软件SPI发送及接收
调试UART RX及TX	P0.2及P0.3	UART0，38400
ESP UART TX及RX	P0.4及P0.5	普通版未启用，只预留配置
ESP RESET	P0.6	普通版未启用
RC522硬件RST	未确认	不操作P1.3，使用软复位

IAR定义OSC32K_CRYSTAL_INSTALLED为FALSE，表示MCU板级外部32k晶振未安装；这与独立RX8010的RTC不是同一器件。查询引脚先看board_config，确认SDK适配再看hal_board_cfg；不要从旧参考代码的GPIO定义反推当前PCB接线。

功能和实现位置总表

功能	主要入口	实际执行位置
上电外设初始化	SAMPLEAPP_INIT_EVT	两端Peripherals_Process，随后调用HAL
自动建网与入网	SDK ZDApp_Init及ZDO事件	ZDApp.c和ZDO模块，角色主文件接收状态
普通刷卡考勤	两端Card_Process	终端Execute及App_Database_Execute
注册注销退出	Key_Process或Command_Process	Begin_Mode、Set_Mode及数据库扫描分支
在线识别	终端Hello，协调器Unicast_Receive	地址、会话号及15秒在线年龄
统一校时	协调器Second_Process或TIME命令	ATT_TIME，终端Set_Time及hal_rtc
查询名单与日志	LIST及LOG命令	Console_Process逐条请求，数据库只读分支
EEPROM可靠提交	DATABASE_EVT	app_database或app_host_store状态机
中文OLED	OLED_EVT及SECOND_EVT	app_ui选择页面，hal_oled分段写入
LED与蜂鸣器	秒任务和结果处理	SDK HalLedSet，独立Hal_Buzzer_Beep
串口收发与解析	UART_RX_EVT及CONSOLE_EVT	hal_console_uart和device1_uart

分阶段阅读顺序

阶段一 先确认编译的是哪个角色

打开CC2530DB/SampleApp.ewp，检查CoordinatorEB和EndDeviceEB的文件排除及编译选项。两个配置分别选择Coordinator.c加app_coordinator加app_host_store，或EndDevice.c加app_terminal加app_database。两端主入口名字相同，但不会在同一个配置里同时编译。

协调器附加f8wCoord.cfg，终端附加f8wEndev.cfg；双方都使用f8wConfig.cfg。角色不是依靠运行时ROLE判断切换。终端配置包含NWK_AUTO_POLL，适配无路由职责的终端下行轮询。

应能回答：为什么SampleApp_Init有两个定义却不重复链接，自己烧录的固件到底是哪个角色。

阶段二 读配置和调度骨架

依次打开board_config.h、SampleApp.h、OSAL_SampleApp.c、Coordinator.c和EndDevice.c。先画出事件表，再对照SampleApp_ProcessEvent中的每一个分支。此时只看业务函数名，暂时不进入驱动细节。

应能回答：事件从哪里来，谁调用ProcessEvent，为什么返回剩余事件，消息什么时候释放，组网在哪里启动。

阶段三 完整跟踪一次终端刷卡

从App_Terminal_Card_Process依次读Request、Execute、App_Database_Execute，再回到DATABASE_EVT读App_Database_Tick和App_Terminal_Database_Process。最后看Show_Result及ATT_EVENT通知。先看普通考勤，再看注册和注销两个分支。

应能回答：读到UID为什么还没有成功，ATT_BUSY如何表示异步提交，落库后如何显示、鸣叫和通知协调器。

阶段四 跟踪协调器读卡和无线应答

从App_Coordinator_Card_Process进入Request、Comm_Protocol_PackFrame、Comm_Unicast_Send。再从终端Unicast_Receive进入Execute。成功应答经协调器Unicast_Receive校验事务后进入Show_Result和本机统计。

应能回答：协调器为什么不能独立离线保存考勤，AF发送成功与数据库提交成功有什么区别，超时后是否可以认定数据没有写入。

阶段五 阅读持久化和时钟

先读app_database.c的镜像、字段和状态，再读hal_eeprom.c的分页及ACK轮询，最后读app_host_store.c。RTC从两端的Time_Payload和Set_Time进入hal_rtc.c，区分年月日的二进制数值、RTC的BCD寄存器及学号压缩BCD。

应能回答：断电时如何选旧库，新库什么时候生效，16位代次和事务号如何回绕，终端没有法拉电容时如何恢复时间。

阶段六 阅读输入输出和底层驱动

读device1_uart.c与hal_console_uart.c，再读app_ui.c、app_font.h及hal_oled.c。最后读hal_rc522.c、hal_i2c.c、hal_board_key.c和hal_buzzer.c。驱动读法是“公开接口 → 参数和返回值 → 有限等待 → 寄存器或总线操作 → 失败退出”。

应能回答：UART为什么不在中断里解析命令，OLED为什么只有128字节缓冲，I2C为什么要上拉，多卡场景有没有完整支持。

启动过程和自动组网

从main到应用任务

SDK的ZMain.c完成硬件、板级和系统初始化，调用osal_init_system，最后进入osal_start_system。系统初始化会调用OSAL_SampleApp.c的osalInitTasks，按tasksArr的顺序注册MAC、NWK、HAL、APS、ZDO和应用任务。

tasksArr里的处理函数顺序必须与初始化时taskID递增顺序一致。否则某个任务ID的事件会分发给错误的处理函数。AF不是独立应用任务，启动路径中有afInit；应用随后用afRegister注册自己的端点。

关键源码：SDK Projects/zstack/ZMain/TI2530DB/ZMain.c的main；Source/OSAL_SampleApp.c的osalInitTasks；SDK Components/osal/common/OSAL.c的osal_run_system。

组网的确切位置

组网由ZDApp_Init调用ZDOInitDevice安排启动。ZDApp_event_loop处理ZDO_NETWORK_INIT，再调用ZDO_StartDevice。角色主文件不会再调用一次建网函数；它在ZDO_STATE_CHANGE消息分支更新SampleApp_NwkState并调用Network_Update。

协调器应用以DEV_ZB_COORD作为网络成功条件，终端以DEV_END_DEVICE作为入网条件。Device_annce只是设备入网公告；它不能证明该设备有本项目的端点、协议或数据库。因此终端地址和业务在线状态仍通过ATT_HELLO确认。

源码定位：SampleApp_ProcessEvent 第78行；SampleApp_ProcessEvent 第76行；SDK ZDApp.c的ZDApp_Init、ZDOInitDevice、ZDApp_event_loop。

网络参数与应用参数

f8wConfig.cfg当前设置DEFAULT_CHANLIST为0x02000000，即只选择信道25；ZDAPP_CONFIG_PAN_ID为0x2B2B。这里的信道掩码不是十进制信道号。组网成功前读到CH为0，不能单凭此值判断配置错误；运行态以协议栈_NIB为准。

应用使用端点20、Profile 0x0F08、Device ID 0x0001。Cluster 1和2保留原示例分类；当前考勤使用单播Cluster 3，广播Cluster 4保留接口但未承载本系统的核心数据库动作。本项目两端配置相同端点，不能推广为所有Zigbee设备必须使用相同端点号。

PAN和信道解决网络选择，端点和Cluster解决应用分发，ATT2操作码解决业务动作，UID解决卡片身份。四层概念不要混用。

OSAL消息和事件调度

这是协作事件调度

osal_run_system先更新时间并轮询HAL，再找到任务表中第一个有事件的任务。它在临界区取走该任务的事件位并暂时清零，调用任务处理函数，然后把返回的未处理事件重新合并回去。

因此一个应用事件不能长时间死等外设。数据库写入等待、OLED全屏输出和串口发送被拆成短步骤；当前I2C和RC522底层仍是有界的同步操作，并不是全部通信都变成硬件异步。OSAL不是本项目中新增的FreeRTOS任务，也不能把10ms周期理解为永远准确的10ms实时期限。

当前事件表

事件	位值	周期或来源	主要工作
SYS_EVENT_MSG	0x8000	SDK消息入队	KEY_CHANGE、AF、ZDO及状态消息
INIT	0x0001	延迟1000ms，步骤100ms	分步外设初始化
KEY_SCAN	0x0002	10ms	SDK采样，项目消抖，业务按键处理
UART_RX	0x0004	SDK UART回调	收完整行并解析
DATABASE	0x0008	10ms，外设初始化后	EEPROM读取、写入、验证和完成处理
CONSOLE	0x0010	10ms	推进TX及逐条帮助、状态、列表输出
OLED	0x0020	10ms	每次最多31列
CARD	0x0040	200ms，外设初始化后	选卡、读UID和业务请求
SECOND	0x0080	1000ms	RTC、模式超时、在线年龄及校时
RETRY	0x0100	单次2000ms	最多4次发送，同一事务重发
BUZZER	0x0200	单次定时	关闭蜂鸣器

表中INIT等缩写均有SAMPLEAPP_前缀和_EVT后缀。初始化时先启动按键、串口维护、OLED和秒任务；数据库及刷卡定时器在Peripherals_Process完成后才启动。g_local_timers_started避免重复创建这些周期定时器。

为什么只处理一段事件就返回

例如events同时包含SYS_EVENT_MSG、CARD和SECOND，应用先清完消息队列，返回events异或SYS_EVENT_MSG，保留CARD与SECOND。OSAL把剩余位重新登记，下一次再调度。这里的异或安全是因为进入分支前已确认该位存在。

同一个事件位重复置位会合并，事件位不是次数队列。如果必须区分多条无线消息，使用SDK消息队列；如果必须保存每条考勤，使用数据库记录，而不是依靠CARD事件出现多少次。

以下为当前刷卡事件分支的实际代码，完整消息处理见源码定位。

```
if (events & SAMPLEAPP_CARD_EVT)
{
    App_Coordinator_Card_Process();
    return (events ^ SAMPLEAPP_CARD_EVT);
}
```

消息为什么必须释放

SYS_EVENT_MSG分支不断osal_msg_receive，逐条switch处理后osal_msg_deallocate。AF帧的frame.data只是借用本条消息缓冲，消息释放后不能保存这个指针。需要重发的内容复制到g_pending_frame；EEPROM写接口也把调用方内容复制到自己的固定缓存。

源码定位：SampleApp_ProcessEvent 第78行；Comm_Protocol_UnpackFrame 第120行；Hal_Eeprom_Write 第68行。

外设初始化及关键状态

初始化顺序和READY的含义

协调器按步骤初始化I2C、RTC、RC522，最后初始化本机统计、OLED和时钟显示。终端初始化I2C、RTC、RC522、OLED，再启动AT24C08数据库。两端外设初始化独立于组网；网络先完成或外设先完成，终端都能在适当时机发HELLO。

g_peripherals_ready表示“应用的初始化步骤已经走完”，不表示每个器件都成功。RC522是否可用看g_reader_ready；RTC是否有效看g_rtc_read_status；OLED看App_Ui_Error；数据库看对应Status函数。一个器件失败不会被READY伪装成正常。

需要先认识的变量

变量	所在模块	读代码时的含义
g_network_joined	两端业务	协议栈角色状态是否满足业务联网条件
g_peripherals_ready	两端业务	初始化步骤是否完成
g_reader_ready	两端业务	RC522初始化结果，决定是否轮询
g_card_held_near_reader	两端业务	读到卡后锁住，检测到无响应后才解除
g_attendance_mode	两端业务	0考勤，1注册，2注销
g_mode_timeout_seconds	两端业务	管理模式剩余秒数
g_terminal_age_seconds	两端业务	上次有效对端活动距今秒数，超过15秒视离线
g_pending_frame_length	两端业务	是否有无线事务或通知等待确认
g_session_id及g_transaction_counter	两端业务	当前启动会话和递增事务号
g_database_mutation_pending	终端业务	数据库写事务尚未最终提交
g_state及g_offset	数据库和EEPROM	当前异步步骤与分块位置，模块各自私有
g_console_output_step	两端业务	帮助或状态输出到哪一条
g_dirty及g_painting	UI	是否需重绘，以及当前是否在分段绘制

static加g_仍然是文件私有全局变量，不会自动变成跨模块公共变量。不同C文件都有g_state，但因static而各自独立。函数内static的seconds或event_tx保持局部作用域，只是生命周期跨越多次调用。

按键 注册 注销和退出

从电平到业务动作

按键1对应P0.1及KEY0_PRESS，按键2对应P2.0及KEY1_PRESS，低电平表示按下。SDK HalKeyRead提供原始状态；Hal_Key_Poll每10ms采样，连续3次稳定后才更新状态，只返回新出现的按下边沿。持续长按不会反复切换模式。

两端Key_Process把边沿转为动作：按键1从非注册模式切到注册，再按退出；按键2对注销模式同理；双键同时按退出；另一键切换管理模式。模式剩余30秒，由SECOND事件逐秒减少，归零回到考勤。

源码定位：Hal_Key_Poll 第14行；App_Terminal_Key_Process 第281行；App_Coordinator_Begin_Mode 第987行。

协调器和终端的模式关系

终端是执行扫描时的模式与注册身份信息权威端。协调器通过ATT_MODE提交模式，终端校验后设置g_attendance_mode和g_registered_identity，并应答。终端本机按键改变模式后通过HELLO带回模式和剩余秒数。

协调器退出先在本机生效。如果在途的是旧ATT_MODE，可以停止该重试并用新事务取消；如果别的事务仍在途，则g_mode_cancel_deferred记录取消意图，待空闲、在线时补发。不要把本机已退出理解为对端取消消息已经送达。

终端收到新的远程ATT_SCAN时，会使用自己的当前模式和注册身份覆盖请求中的对应部分；重复事务则恢复原执行模式。应答尾部携带实际扫描模式，避免协调器因并发的终端事件显示错误的“注册成功”或“考勤成功”。

注册与注销到底写了什么

注册不写RFID卡扇区，只把卡UID绑定到EEPROM中的用户槽。数据库先查重复UID，再找空槽。无指定身份时按单调递增计数生成12位学号，从202600000001开始；姓名为Student加槽号对应数字。指定身份时校验12位数字、1到15个ASCII姓名字符和学号唯一性。

注销把40字节用户槽清零，不删除历史日志。已有考勤保存学号快照，注销或改名不会把过去的记录变成新用户的记录。注册成功、重复卡、注销成功、未注册卡及满库是不同业务结果，不能只用一个“OK”概括。

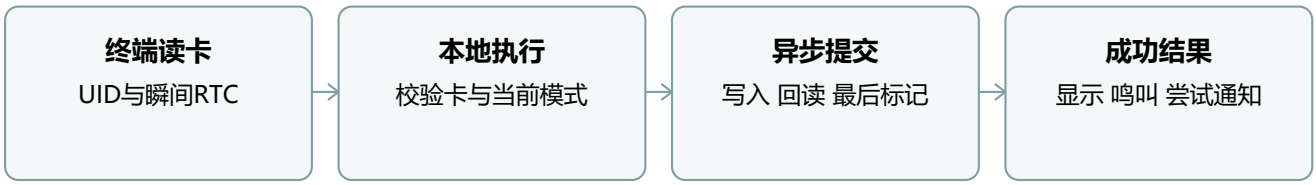
源码定位：App_Database_Execute 第320行；App_Database_Identity 第289行。

两条考勤链路

终端本地刷卡链路

CARD事件调用Card_Process。RC522成功返回UID后，先进行持卡门控，保存最近UID并检查是否有数据库写事务。随后组织48字节扫描载荷，读取刷卡瞬间RTC，填入时间有效标记和origin为1，清除本次管理模式，调用Request。

终端Request直接调用Execute，不绕无线发送。Execute调用App_Database_Execute；只读或参数错误可立即返回，写操作返回ATT_BUSY并登记待完成事务。DATABASE事件持续推进Tick，最终提交后才通过Complete_Reply取得结果、显示OLED、鸣叫并决定是否向协调器发ATT_EVENT。



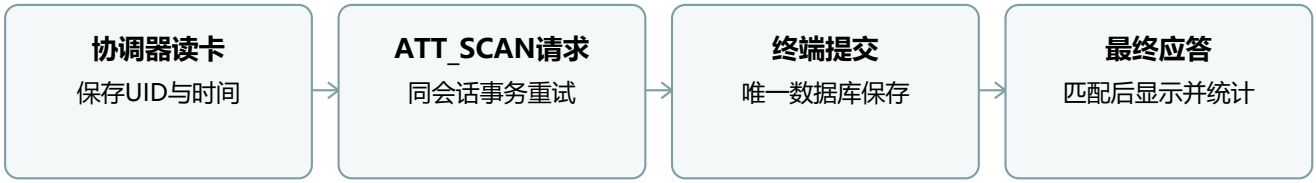
源码定位：App_Terminal_Card_Process 第1017行；App_Terminal_Execute 第829行；App_Terminal_Database_Process 第196行。

协调器读卡链路

协调器读取UID和自己的RTC，origin为0。Request要求网络已建立、业务终端在15秒内在线、没有在途请求；通过ATT2组帧并保存重发副本，再由Comm_Unicast_Send送到终端短地址。

终端只接受来源短地址0的业务帧，校验后执行同一个数据库逻辑。持久化成功才回复ATT_SCAN加ATT_REPLY。协调器必须匹配来源地址、操作码、会话、事务号及最小结果长度；ATT_BUSY只表示继续等待，最终结果才停止重试。

收到本机刷卡成功应答后，协调器使用原请求中捕获的UID和时间更新AT24C02本机统计，再显示结果。统计存储故障单独报告，不撤销终端已成功保存的考勤。终端本机刷卡事件不会计入协调器本机刷卡统计。



源码定位：App_Coordinator_Request 第780行；App_Coordinator_Unicast_Receive 第810行；App_Host_Store_Record 第121行。

持卡抑制和忙状态

持卡抑制并非比较UID是否相同，而是成功读到卡后设置g_card_held_near_reader；只有RC522报告NO_RESPONSE才解除。直接换另一张卡但没有观察到无卡间隙，也可能被抑制。因此演示每次都应移开卡再刷。

数据库忙、协调器已有在途事务或本机统计忙时，会提示BUSY及移卡重试，不建立无限队列。被忙状态拒绝的这次卡仍需移开后重新触发。

离线和超时的实际行为

终端离线仍可提交本地考勤。通知协调器失败只是通知失败，日志继续存在；已有ATT_EVENT待确认时不阻止新的本地落库，但新结果未必再建立另一条实时通知。重连HELLO同步概要，详细历史通过LOG查询。

普通版协调器没有离线考勤队列。远程请求超时也不能证明终端没有写入：可能是已提交但应答丢失。正确做法是查询LOG或LIST核对，避免把“未确认”说成“绝对没有落库”。

无线协议和事务可靠性

AF消息如何进入业务

AF_INCOMING_MSG_CMD进入SampleApp_MessageMSGCB，按clusterId分到Comm_Unicast_ProcessMSG或广播模块。Common从SDK包取源短地址、Data和DataLength，再同步调用注册的角色接收回调。业务调用UnpackFrame后才读取ATT2字段。

发送则反向经过角色Send或Request、PackFrame和Comm_Unicast_Send。AF_DataRequest使用16位目的地址、端点20及单播Cluster 3。返回COMM_OK表示请求被协议栈接受，不代表RFID成功或EEPROM提交完成。

源码定位：Comm_Unicast_Send 第54行；Comm_Unicast_ProcessMSG 第109行。

ATT2帧结构

字节偏移	长度	字段	含义
0	1	帧头	0xA7
1	1	版本	2
2	1	opcode	动作，响应加0x80
3	1	data_len	载荷长度，最多52
4	2	session	小端启动会话
6	2	transaction	小端事务号
8	N	data	业务载荷
8加N	2	CRC16	小端，覆盖前面的帧头至数据

总帧长为N加10，最大62字节。校验顺序是空指针、总体长度、帧头及版本、载荷长度一致性、CRC；验证后才把data指针交给业务。CRC初值0xFFFF，多项式0x1021；它检测错误，不进行加密或身份认证。

所有业务操作码

值	名称	作用
1	SCAN	按当前模式考勤、注册、注销
2	USER	查用户槽
3	LOG	查日志索引
4	SET	修改学号姓名
5	DELETE	清除用户槽
6	FORMAT	明确初始化数据库
7	META	数据库状态及容量概要
8	MODE	管理模式和注册身份
9	TIME	协调器时间
10	HELLO	业务握手、人数、模式、在线确认
11	EVENT	终端已完成业务的通知
12	FIND	按UID查用户

基本响应为status、slot及16位gen，共4字节。USER完整响应44字节，LOG为20字节，META为8字节。SCAN还附加实际模式，因此成功带用户信息时可为45字节；注销等无用户详情或重放响应通常为5字节。

扫描载荷必须会解释

载荷偏移	长度	内容
0	1	模式0、1、2
1	1	UID长度4、7、10
2	10	UID固定预留区，未用部分为0
12	12	学号ASCII，自动身份时为空
24	16	姓名及结尾0
40	6	年月日时分秒，普通二进制数值
46	1	时间有效标记
47	1	origin，0协调器，1终端

TIME载荷也是6个普通数值；RTC寄存器才是BCD。学号在用户表是12字节ASCII，在考勤日志中压缩成6字节BCD。不要因为都叫“日期或数字”就把编码混在一起。

HELLO 在线与同步

终端每5秒发7字节HELLO：数据库status、人数、gen两字节、RTC状态、模式、剩余秒数。协调器记录源地址和终端启动会话，清零在线年龄，并回一个HELLO。终端收到协调器有效帧时也清零自己的对端年龄。

15秒业务在线判断与Zigbee入网状态是两回事。网已建立但终端没运行本项目业务时，协调器仍显示终端离线。终端RTC状态非零、会话改变、地址改变或长时间断联重来时，协调器安排校时。

重试和防重复的三个层次

第一层是重发同一帧：首次发送把retry_count设为1，每2秒最多继续到4次，不改会话、事务、UID或刷卡时间。第二层是终端RAM缓存最后一条操作及结果，重复的MODE、TIME或写操作可以重发结果而不重做。第三层是AT24C08头部保存最后已提交远程修改的会话、事务、操作、槽位、结果和模式，终端重启后仍能识别该修改。

16位事务比较采用差值半区间方式，允许自然回绕。保护窗口不是无限历史：持久化只记录最后远程修改，新的会话是另一组请求；不能据此宣称恶意重放或断电下所有任意历史请求都绝对防重。

源码定位：App_Database_Request_Fresh 第90行；App_Terminal_Unicast_Receive 第896行；App_Coordinator_Retry_Process 第186行。

AT24C08数据库和掉电恢复

容量为什么是8人和10条

AT24C08共1024字节，分成两个512字节镜像。每份32字节头部，加8个40字节用户槽，再加10个16字节日志，正好512字节。两个镜像提供更新失败时保留旧库的空间。

区域	单镜像偏移	字节数
头部	0到31	32
用户表	32到351	320
日志表	352到511	160

镜像0在EEPROM 0x000到0x1FF，镜像1在0x200到0x3FF。RAM中只有一份g_db的512字节镜像，另有32字节g_check和44字节g_reply。比较两个镜像时依次读取并保存有效性和代次，随后装载选中的镜像；没有为每个镜像各占512字节RAM。

头部字段

偏移	长度	作用
0	4	ATD2魔数
4	1	版本2
5	1	提交标记0xA5
6	2	数据库代次gen
8	1	下一条日志写槽
9	1	已有日志数量，最多10
10	2	自动学号计数，空库初值1
12	2	上次远程修改session
14	2	上次远程修改transaction
16到19	4	上次操作、槽位、结果、有效标记
20	2	考勤序号
22	1	上次远程扫描实际模式
23到29	7	保留区
30	2	镜像CRC16

计算镜像CRC时暂时把提交标记和CRC字段置0，计算512字节后恢复。因此最终把标记从0写为0xA5不会导致CRC与已提交镜像不一致。

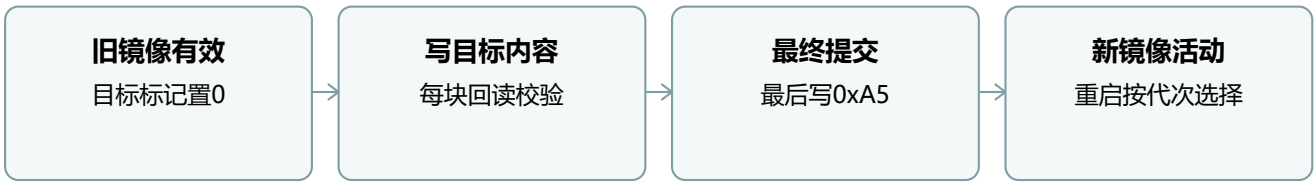
用户槽与日志槽

40字节用户槽依次包含UID长度1字节、UID预留10字节、学号12字节、姓名16字节、标志1字节。UID长度为0视为空槽，支持长度4、7、10。姓名区最多15个字符加结尾0。

16字节日志依次包含压缩学号6字节、年月日时分秒6字节、valid1字节、origin1字节、16位序号2字节。日志保存学号而不保存姓名，也不保存整段UID；注销后历史身份仍可通过学号快照解释。

日志写入位置g_db[8]每次加1并对10取模；数量g_db[9]增加到10后不再增长。读取最新第i条时使用“下一写槽加10减1减i再对10取模”。第11条会覆盖最早一条，这不是无限历史数据库。

写入状态机



状态	工作	为什么要这样做
1、2	各读一个镜像，每次32字节	判断有效性和代次，不申请两份大RAM
3	重新装载选中或失败后的旧镜像	RAM与有效EEPROM保持一致
4、5	目标提交标记置0并等写完	明确目标还不可信
6、7	写32字节，等完成并回读比较	逐块确认写入内容，底层再按页拆分
8、9	最后写0xA5，等完成并读回	只有全部完整才提交新库
0	空闲或错误状态	允许查询及下一事务

App_Database_Execute成功修改RAM后返回ATT_BUSY，表明已经进入异步提交。Tick完成时设置g_done；Take_Done读取并清除这一次完成标志，终端业务随后取最终回复。业务不能在Execute返回ATT_BUSY时就显示注册成功。

上电和中途掉电

上电检查魔数、版本、提交标记、CRC和用户字段合理性，选有效且代次较新的镜像。两个镜像都全FF时建立RAM空库；普通空库并不是在上电时立即写整片EEPROM。非空但两个镜像均无效则报CORRUPT，不自动清空。

写目标途中掉电，旧活动镜像未动，新目标标记无效；重启选旧库。最终标记完成之后掉电，重启可能选新库。实际掉电和总线电气仍需实板验证，软件测试覆盖许多切断步骤不等于证明任何硬件故障都能恢复。

写入失败后App_Database_Failed重新装载旧镜像，回复ATT_STORAGE，避免把未持久化的RAM修改当作成功记录供查询。

源码定位：App_Database_Tick 第130行；App_Database_Image_Valid 第28行；App_Database_Failed 第111行。

EEPROM驱动和协调器本机统计

AT24C08如何寻址和分页

Hal_Eeprom_Device把地址高位放入7位器件地址，0x50到0x53对应AT24C08四个256字节块，低8位为块内字地址。单镜像跨过0xFF时必须切换器件地址，不能只让8位字地址自然回绕。

AT24C08配置1024字节容量、16字节写页；AT24C02配置256字节容量、8字节写页。Read按32字节及256字节块边界拆分；Write接受最多32字节并拷贝到驱动缓存；Process再按页边界发送，随后用空写探测ACK。

写页后立即NACK可能表示芯片内部写周期未结束。Process返回I2C_BUSY，让OSAL以后继续；超过20ms仍NACK转为TIMEOUT。这个20ms是代码超时阈值，不是每次写入必须耗时20ms。

源码定位：Hal_Eeprom_Device 第11行；Hal_Eeprom_Process 第94行。

AT24C02保存哪些统计

协调器使用两个64字节HST1镜像，仅占前128字节，后128字节不写。头部含魔数、版本、提交标记、代次；偏移8是启动次数，10、12、14分别为本机考勤、注册、注销成功次数；16为最近UID长度，17到26为UID，27到32为时间，33为时间有效标记，62到63为CRC。

启动时选择有效镜像并异步增加启动次数。Record仅在本机远程扫描收到成功应答后发起本机统计保存，计数达到65535后饱和，不回绕成0。Record返回ATT_OK表示接受本次统计保存；真正完成仍由Process推进。

本机统计不是两端总考勤次数，也不是终端日志条数。终端循环日志只保留10条，而本机统计可累计更多次；清终端数据库不会自动等同于清协调器统计。

RTC统一时间与有效性

协调器为什么是唯一时间源

协调器RTC有效时直接使用现有时间；无效时用本次固件__DATE__和__TIME__初始化，并回读确认。这是编译时间，不是烧录时间、网络时间或每次上电的真实当前时间。准确演示前应在协调器输入实际北京时间。

终端不调用Seed_Build_Time，也不允许本机串口单独设时间。入网和外设就绪后发HELLO，协调器在秒任务中安排ATT_TIME；在线时约每60秒校准。终端写RTC并立即回读，然后RTC硬件自行走时，CPU每秒只是读取并显示。

TIME命令手动同步时，终端Hello传time_sync_request为1，使既有RTC状态字段非零表达取时请求，既不清除本机有效时间，也不另造时钟。传输和任务调度存在延迟，当前实现没有毫秒级往返时延补偿。

源码定位：App_Coordinator_Second_Process 第1212行；App_Terminal_Hello 第794行；App_Terminal_Set_Time 第760行。

RX8010驱动实际做什么

驱动使用7位地址0x32，从0x10读秒到年的7个寄存器。先检查0x1E的VLF和0x1F的STOP，再检查BCD、星期单一位编码和年月日范围。年份范围为2000到2099，闰年规则在这个范围内使用year对4取模。

设置时先停止RTC，写BCD时间、星期位及控制相关默认值，再释放STOP。无论中途哪一步报错，都会尝试释放STOP并保留原始错误。驱动初始化探测状态，不默认覆盖有效RTC。

时间状态0才有效，5表示无效时间；I2C错误也可能直接返回。刷卡瞬间再次读RTC，记录valid为0时，时间字段可能是此前缓存值，不能当真实有效打卡时间，也不会在后来校时后自动补成准确时间。

终端没有法拉电容，因此不能依赖RTC掉电保时。保持供电的无线离线状态可以继续走时；RTC真的掉电后需重新取时。

串口命令与非阻塞输出

唯一串口拥有者

hal_console_uart负责UART0打开、收发缓冲、拼行和背压。Common的Device1_UART_Init只重置解析回调，不再次打开UART。角色主文件注册完整行处理函数及业务命令回调。

串口为38400、8N1、无流控。当前普通版有256字节TX环形缓存，留一个空位区分满与空，所以最多255字节有效容量；SDK RX和TX配置各128字节。每次接收最多32字节，单行缓冲80字节，可接受79个ASCII字符。

UART回调只置事件。任务中收齐CR、LF或CRLF后才解析；支持退格，过长或非法字节整行丢弃并报错。TX每个CONSOLE事件最多推进32字节；SDK写缓冲不足时保留剩余数据，不死等硬件发完。g_uart_dropped统计应用发送缓存丢字节，不等于无线丢包数。

解析阅读顺序

UART_FindCMD在行中定位第一个井号并检查行尾，ParseCMD把命令体复制到固定80字节解析缓存，按空格、TAB或冒号拆分，最多5项。完整分词后UART_GetCMDType识别HELP、STATUS、LIST、LOG、TIME、DB；DB部分动作会移动参数指针复用业务位置，不新建一套通用命令框架。

uart_cmd_t里的arguments指向可复用解析缓存，只在本次同步回调内有效；不能保存指针给以后事件使用。需要跨事件保留的注册信息被复制到g_registered_identity。

源码定位：Device1_UART_ParseCMD 第191行；UART_GetCMDType 第83行；Hal_Uart_Process_Rx 第169行。

当前所有对外命令

命令	用途和限制
HELP	四条常用查询及按键提示
HELP TIME或HELP DB	对应维护帮助
STATUS	网络和外设分两条输出
LIST或LIST 0	全部用户槽或指定0到7槽
LOG或LOG 0	全部日志或最新第0到9条
LOG CSV	CSV表头及日志，最后END
TIME	当前设备RTC状态与时间
TIME YYMMDDhhmmss	仅协调器校时
TIME SYNC	同步协调器时间
DB STATUS	终端数据库概要
DB FIND UIDhex	按4、7、10字节UID查询
DB SET slot ID Name	改学号姓名，保留UID
DB DEL slot	注销指定槽
DB REG或DB REG ID Name	自动身份或指定身份的刷卡注册模式
DB UNREG或DB CANCEL	注销模式或立即退出
DB UID或DB BEEP	最近UID或蜂鸣器诊断
DB INIT CONFIRM	明确重建终端数据库
DB STATS	仅协调器，本机统计
DB STATS RESET CONFIRM	仅协调器，重建本机统计

实际输入每条命令前加井号，例如#LIST。单独的#REG、#RTC、#USER、#GET、#EXPORT及#TIME等号旧写法已合并，不再作为独立命令识别。INIT和RESET仍需显式CONFIRM；不能为了答辩随意执行清库。

HELP、STATUS和列表通过g_console_output_step等状态分条输出；列表一次请求一个槽，等待应答后递增。协调器等待事务期间暂停帮助和状态输出，避免与查询响应争用发送空间。成功查询只输出详情和gen，失败或写动作保留RSP状态。

OLED 中文页面与增量刷新

页面内容由谁决定

app_ui保存角色、网络、对端在线、数据库、人数、模式、秒数和RTC快照。Render生成当前页。屏幕128乘64，分8页，每页8行像素；中文16乘16跨两页，所以业务画面按4条16像素高逻辑行安排。

顶部反白显示角色、组网状态和连接图标。第二逻辑行显示大时钟或操作结果；第三行显示日期、人数、学号或提示。底栏优先显示管理模式倒计时和退出提示，之后依次考虑数据库忙、错误、网络或对端离线，最后显示考勤及按键提示。

管理模式倒计时为30秒；结果页面保持5秒，来自g_message_seconds。这两个计时器不是同一个变量。离线刷卡提示的结果行显示剩余秒数，底栏仍显示对端离线。

源码定位：App_Ui_Render 第209行；App_Ui_Message 第367行；App_Ui_Second 第375行。

中文字模和名字显示

app_font.h是从Noto Sans SC生成的页面用GBK字模子集，每个字32字节点阵，按GBK编码排序，通过二分查找定位。它不是把整套中文字体库全部装入CC2530；需要新汉字时应修改生成脚本的文案并重新生成。字体许可在assets/fonts/OFL.txt。

汉字每个16列，ASCII每个6列；大时钟用5乘7数字字模放大。界面文案是中文，但数据库姓名仍限制ASCII，不能据此宣称支持中文姓名注册。hal_oled中的旧Show_Line接口用于ASCII行，当前中文业务主要通过Write_Chunk发送像素。

为什么只有128字节缓冲

UI只保存g_pixels的单页128字节，不保存1024字节整屏。Tick从页0、列0开始，每次最多31列，页尾后进入下一页。128列需要5次发送，8页共40个OLED事件；若事件持续按10ms调度，一轮像素输出约400ms，实际还受任务及总线耗时影响。

脏标记表示需要新一轮绘制，不等于每次调用都立即刷新整屏。绘制期间新状态可以更新；当前实现没有整帧原子快照，切换瞬间可能看到分段更新。OLED错误后Tick不再继续写，需要重新初始化或重启处理。

SSD1306初始化探测0x3C，再在NACK时探测0x3D，使用页寻址及当前0xA1、0xC8方向设置。像素写入控制字为0x40，命令控制字为0x00；31列加控制字正好适配32字节I2C传输上限。

底层器件逐个阅读

I2C总线

SCL为P1.2，SDA为P1.1。GPIO输出锁存器始终为0，切输出方向拉低，切输入方向释放，依靠外部上拉得到高电平；不能将释放理解为主动推挽输出1。OLED、RTC和EEPROM共用总线但地址不同。

Start在SCL高时把SDA拉低；Stop在SCL高时释放SDA；写字节从最高位开始，随后释放SDA读ACK；读字节最后向从机发送ACK或NACK，最后一个字节用NACK。Read_Reg先写寄存器地址，再产生重复起始读数据。

初始化最多送9个恢复时钟并产生Stop；SCL释放最多轮询100次，每次调用5微秒等待。它是有界恢复，不能修复GND虚接、供电错误或从机持续短路。地址使用7位值，在发送时才左移加读写位。

源码定位：Hal_I2c_Init 第46行；Hal_I2c_Read_Reg 第237行。

RC522和RFID

RC522使用P1.4到P1.7软件SPI。寄存器地址左移后加读位，CS低选中，CLK上升沿采样MISO。初始化检查版本0x91、0x92或0x88，软复位并配置定时器、模式和天线。未确认的硬件RST不操作。

读卡链为WUPA唤醒，CL1到CL3的防碰撞响应、BCC校验、SELECT和SAK的CRC_A校验，根据Cascade Tag去掉0x88，累计4、7或10字节UID，最后HALT。HALT正常不回应，函数会恢复成功状态；下一轮WUPA仍能唤醒同卡。

当前实现是单卡课程演示选卡路径，碰撞或响应长度不符返回错误，没有实现任意位冲突的多卡仲裁树。也没有MIFARE扇区认证、读写扇区或UID防克隆机制。

Exchange最多轮询400次，每次有10微秒等待及SPI寄存器读取；不能把400乘10微秒当完整函数的精确4ms耗时。

源码定位：Hal_Rc522_Read_Uid 第256行；Hal_Rc522_Exchange 第152行。

LED和蜂鸣器

LED是P1.0，高电平亮；SDK HalDriverInit完成初始化，应用秒任务在网络及业务对端就绪时常亮，否则每秒Toggle。Toggle是翻转一次电平，不是自己启动新的闪烁线程。

蜂鸣器是P0.0，高电平响。Beep先关闭、停止旧定时器并清旧事件，成功安排关闭事件后才打开；定时器申请失败则保持静音。成功提示通常60ms，错误通常180ms；诊断BEEP是80ms。关闭由角色主文件BUZZER事件调度。

源码定位：Hal_Buzzer_Beep 第29行；SDK hal_led.c的HalLedSet及hal_key.c的HalKeyRead。

延时接口与未参与主流程的接口

hal_delay.c保留原NOP忙等接口，其循环不能等同于经过仪器校准的微秒。当前主要总线驱动使用SDK MicroWait；不要为了保留旧文件就把长Hal_Delay_Ms加回任务处理函数。附录标出主业务未引用的接口，供读者区分“文件里存在”与“正常运行会调用”。

返回值 异常和资源约束

同一个数字在不同模块可能不同

域	数值	含义
ATT业务	0到9	OK、BUSY、PARAM、UNKNOWN、FULL、EXISTS、STORAGE、CORRUPT、OFFLINE、TIMEOUT
I2C	0、1、2、3、4	OK、NACK、TIMEOUT、PARAM、BUSY
RTC	0或5，以及I2C错误	有效时间或TIME_INVALID，以及总线错误
RC522错误变量	0到4	OK、NO_RESPONSE、PROTOCOL_ERROR、NOT_READY、BAD_PARAM
RC522 bool返回	1或0	成功或失败

DB BUSY是1，I2C BUSY是4；I2C状态2表示超时。解析日志时先确认模块前缀，不能把业务错误表套到所有驱动。

Hal_Eeprom_Write返回0表示请求已接收，之后要Process确认完成。App_Database_Execute返回ATT_BUSY可能表示已启动提交，需结合调用前是否空闲和后续Done判断。Comm_Unicast_Send成功只表示AF接收发送请求。

内存和性能为什么这样安排

TI资料给出CC2530的增强8051、8KB RAM及相应Flash配置；当前工程使用banked链接布局。代码占用超过64KB不能仅凭传统8051直接寻址范围就判断不可运行，应查看IAR的银行和链接配置。TI CC2530资料

本项目链接区XDATA从0x0001到0x1EFF，共7935字节，不把整片8KB都当应用自由空间。2026年10月8日验证记录中普通版协调器XDATA为7636字节，终端为6571字节；CODE分别165170及136941字节，CONST分别6724及6729字节。静态链接余量不等于运行时任务栈和OSAL堆的安全余量。

应用采用固定缓存、单个在途事务、分页显示及分块提交。协议栈内部仍会动态分配消息、定时器及AF资源，不能宣称系统完全不用堆。8051参数和可变参数打印需注意宽度，源码把%u和%x参数显式转为unsigned int，轻量Printf只实现%s、%c、%x、%u，不能随意用浮点或标准printf全部格式。

当前设计边界

本实现没有迟到早退或签到时段规则，没有无限日志、多业务终端映射、中文姓名输入、离线协调器队列、跨设备同时刷卡排队、强一致时延补偿和完整多卡仲裁。SECURE为0，UID不等于不可伪造凭据；不应把课程演示系统介绍成已经具备生产门禁安全能力。

答辩演示和验证依据

推荐现场演示顺序

1. 先展示两个IAR角色配置、board_config、SampleApp.h和ProcessEvent，解释任务在哪里调度。

2. 上电后等待初始化及数据库就绪，用#STATUS看网络、外设和业务对端，用#TIME看RTC有效性。

3. 在协调器设置准确时间，示例#TIME 261013103000表示2026年10月13日10点30分，不是自动获取的当前时间；终端#TIME SYNC后核对#TIME。

4. 按键1，刷白卡，移卡，#LIST显示注册信息；再按注册键刷同卡演示重复注册。

5. 普通模式用终端刷卡，#LOG核对学号、时间、valid及origin为1；再用协调器刷卡，检查origin为0。

6. 进入管理模式后同键再按，演示立即退出，不必等30秒；再展示超时退出机制代码。

7. #LOG CSV展示记录导出；#DB STATS展示协调器本机统计，并解释它与终端记录数不同。

8. 按键2注销，#LIST及普通刷卡提示未注册；再查#LOG确认旧历史没有删除。

9. 若时间和设备允许，演示终端无线离线仍可保存、重新上电读EEPROM。实际断电提交测试需先备份名单，勿在正式演示中临时清库。

演示代码讲解抓住一个具体UID、一条事务号、一条EEPROM记录。不要只指着大段函数说“这里处理业务”；要说明输入、状态和输出。

已有软件测试对应什么

测试	验证的核心行为
test-board-hal	蜂鸣器安全关闭、定时失败；按键消抖和边沿
test-storage	EEPROM分页、超时；RTC BCD、日期及VLF
test-host-store	AT24C02统计、重启、60个中断提交场景
test-rc522	SPI帧、4/7/10字节UID、BCC和CRC及错误
test-attendance	AT24C08镜像、110个断电步骤、重放、满库、循环日志
test-app	实际两端业务、事件组合、事务匹配和按键
test-app加RealUart	实际串口驱动及业务、发送背压、状态和查询交错
test-console	CR/LF、长输入、退格、环形缓冲和发送背压
test-common	AF路由参数、ATT2固定字节、CRC及六组串口解析
test-ui	中文页面、倒计时、边界及14个页面

构建入口为scripts/verify-attendance.ps1。最近现有验收记录在docs/build/readability-delivery.json，日期为2026年10月8日；其中三个版本六个角色配置均零错误零警告，未烧录。本文写作不把历史软件记录改称2026年10月13日的实板验证。

答辩问题和回答依据

问题1 用一分钟介绍这套系统

我用两块CC2530组成一台协调器和一台终端的Zigbee考勤系统。两端都能读RFID卡，人员名单和考勤统一保存在终端AT24C08。协调器管理网络和统一时间，终端校时后由自己的RTC走时。按键支持注册、注销和立即退出，串口支持名单及记录查询，OLED显示中文结果。数据库采用双镜像和最终提交标记，无线用事务号重试及防重复。

源码依据：SampleApp_ProcessEvent 第78行；App_Database_Execute 第320行。

问题2 主程序在哪里 为何两个文件都有SampleApp_Init

真正的main在SDK ZMain.c。应用任务由OSAL_SampleApp.c注册，主任务调度分别在Coordinator.c和EndDevice.c。IAR两个配置各选一个角色文件，因此两个同名入口不会同时链接。业务进入app，GPIO和总线进入hal。

源码依据：SampleApp_Init 第47行；SampleApp_Init 第46行。

问题3 组网具体在哪个函数里启动

SDK ZDApp_Init调用ZDOInitDevice安排网络启动，ZDApp_event_loop处理ZDO_NETWORK_INIT并调用ZDO_StartDevice。应用只注册端点、处理ZDO_STATE_CHANGE，不另建第二套组网。

源码依据：SampleApp_Init 第47行；SampleApp_ProcessEvent 第78行。

问题4 Zigbee已经入网 为什么OLED还可能显示终端离线

入网状态属于协议栈，业务在线还需要ATT_HELLO握手。协调器根据HELLO确认终端地址和会话，并用15秒在线年龄判断。只有网络状态还不足以证明终端数据库业务在运行。

源码依据：App_Coordinator_Unicast_Receive 第810行；App_Coordinator_Second_Process 第1212行。

问题5 PAN 信道 端点 Cluster和UID分别是什么

PAN与信道选择无线网络，端点定位设备上的应用，Cluster区分消息类别，ATT2操作码区分考勤业务，UID标识卡片。本工程PAN为0x2B2B，信道25，端点20，单播Cluster为3；UID与Zigbee短地址没有同一含义。

源码依据：SampleApp_MessageMSGCB 第198行；Comm_Unicast_Send 第54行。

问题6 为什么使用OSAL 而不是在while循环里轮询所有业务

沿用Z-Stack原有协作事件调度，可以与MAC、NWK、APS和ZDO共享事件系统。应用处理短事件后返回，避免长时间占用CPU。外设同步事务仍有耗时，因此周期值不是硬实时保证。

源码依据：SampleApp_ProcessEvent 第78行；App_Database_Tick 第130行。

问题7 同一时刻有多个事件会丢吗

ProcessEvent处理当前优先分支后返回剩余事件，OSAL会重新合并并继续调度。不同位不会因为一次return都消失；但同一个位被反复置位会合并，不是次数队列。无线多消息由消息队列保存。

源码依据：SampleApp_ProcessEvent 第78行。

问题8 为什么消息处理后一定要deallocate

SDK消息占用动态内存，处理后不释放会逐渐耗尽OSAL堆。解包后的data指针只借用消息内容，释放后不能再使用，因此需要跨事件的数据必须复制到固定缓冲。

源码依据：SampleApp_ProcessEvent 第78行；Comm_Protocol_UnpackFrame 第120行。

问题9 g_peripherals_ready是否说明所有外设已经正常

不是。它只表示初始化步骤完成。RC522看g_reader_ready和错误变量，RTC看状态及有效标志，OLED看App_Ui_Error，数据库看Status。初始化不会把某个器件失败伪装成成功。

源码依据：App_Coordinator_Peripherals_Process 第103行；App_Terminal_Peripherals_Process 第112行。

问题10 static和g_有什么作用 为什么不同文件有相同g_state

g_是阅读规范，表示文件作用域变量；static才限制链接可见范围。数据库、统计和驱动的g_state各自属于自己的C文件，不会相互冲突。函数内static则是局部名字、持久生命周期。

源码依据：App_Database_Tick 第130行；App_Host_Store_Process 第170行。

问题11 按键怎么消抖 为什么长按不反复触发

SDK读取GPIO，项目每10ms采样，连续3次稳定后更新状态。pressed等于raw与旧稳定状态的反位相与，仅取新按下边沿。释放需要稳定后才能再次产生按下事件。

源码依据：Hal_Key_Poll 第14行。

问题12 怎样立即退出注册 为什么不用等30秒

同一个模式键再按或双键一起按就切回模式0，串口DB CANCEL也支持退出。协调器退出先在本机生效；若别的无线事务仍在途，会登记deferred，空闲后补发取消。30秒倒计时只作为自动退出补充。

源码依据：App_Coordinator_Begin_Mode 第987行；App_Terminal_Key_Process 第281行。

问题13 两边同时切换模式 以哪边为准

终端在真正执行新扫描时使用自身模式和注册身份，协调器通过ATT_MODE及HELLO与它同步。扫描应答还带实际模式，协调器显示结果时优先使用该事务返回的模式，防止并发事件覆盖。

源码依据：App_Terminal_Unicast_Receive 第896行；App_Coordinator_Unicast_Receive 第810行。

问题14 白卡注册时往卡里写了什么

没有写卡扇区。只读UID并在EEPROM中创建UID、学号和姓名的绑定。RC522代码没有扇区认证或写块接口，因此注册是数据库登记，不是修改RFID卡内容。

源码依据：Hal_Rc522_Read_Uid 第256行；App_Database_Execute 第320行。

问题15 怎样避免重复卡和重复学号

注册前按UID查槽，已存在返回EXISTS；身份校验同时检查12位数字、姓名长度及其他有效槽的学号。自动编号计数单调递增，注销后复用槽位也不会简单复用旧学号。

源码依据：App_Database_Find 第276行；App_Database_Identity 第289行。

问题16 注销后历史考勤为什么还能查

注销只清零用户槽，不清日志。日志中存的是当时学号的6字节BCD快照，不依赖今天这个槽位里是谁，因此后续改资料或复用槽不会改写过去的学号。

源码依据：App_Database_Execute 第320行。

问题17 终端刷卡全过程怎么走

CARD事件读UID及瞬间RTC，Request直接进入本地Execute，数据库在RAM组织修改并启动异步提交。DATABASE事件写入、回读及最终提交后，业务才显示、鸣叫并尝试通知协调器。

源码依据：App_Terminal_Card_Process 第1017行；App_Terminal_Database_Process 第196行。

问题18 协调器刷卡为什么必须等终端应答

终端是唯一考勤数据库，协调器把扫描信息发给它。AF返回成功只说明接受发送，不代表终端完成EEPROM。匹配最终应答后才能确认业务成功，并更新协调器本机统计。

源码依据：App_Coordinator_Request 第780行；App_Coordinator_Unicast_Receive 第810行。

问题19 卡一直放在读卡器上怎么避免重复打卡

读到卡后设置持卡锁，只有下一次RC522报告无响应才解除。不是单纯比较两次UID。忙或未知卡后也要移开重刷；直接换卡但没有无卡间隙可能仍被抑制。

源码依据：App_Terminal_Card_Process 第1017行。

问题20 读到UID为什么还可能显示未注册

RC522读卡成功只是硬件层取得UID，数据库再判断是否有对应用户。普通考勤或注销时未知UID返回ATT_UNKNOWN。硬件成功与考勤业务成功分层，不应混成一个返回值。

源码依据：Hal_Rc522_Read_Uid 第256行；App_Database_Execute 第320行。

问题21 断网后还能考勤吗

终端可以，因为数据库在本机。协调器没有离线保存队列，无法确认远程考勤。终端RTC仍供电且有效时继续记录时间；真的掉电后未同步的时间必须标为无效。

源码依据：App_Terminal_Request 第889行；App_Coordinator_Request 第780行。

问题22 两端同时刷卡会排队吗

没有无限排队。终端一次只处理一个数据库变更，冲突请求返回BUSY；协调器也只有一个在途事务。提示忙后需要移卡重新操作，当前实现不能宣称并发刷卡一定全部自动排队入库。

源码依据：App_Terminal_Execute 第829行；App_Coordinator_Card_Process 第926行。

问题23 通知协调器失败 会撤销考勤吗

不会。ATT_EVENT只在本地提交后通知，重试超时清通知状态并输出日志，不删除记录或把本地成功改为失败。重连后概要可握手同步，完整记录通过LOG查询。

源码依据：App_Terminal_Database_Process 第196行；App_Terminal_Retry_Process 第260行。

问题24 发送超时是否一定表示终端没有保存

不一定，可能是终端已提交但应答丢了。重试使用同一事务防止重复；最终超时表示未确认，应该查询数据库核对，再决定是否重新刷卡。

源码依据：App_Coordinator_Retry_Process 第186行；App_Database_Request_Fresh 第90行。

问题25 为什么重试不重新读取UID和时间

重试的是同一g_pending_frame，内容及会话事务不变。重新采样可能改变时间或操作含义，让一次刷卡被当成多个新请求。首次加三次重发共4次，理想调度下在约8秒判超时。

源码依据：App_Coordinator_Request 第780行；App_Coordinator_Retry_Process 第186行。

问题26 怎样知道收到的应答属于当前请求

协调器同时检查源短地址、opcode带响应位、会话号、事务号及最小响应长度。错误来源或迟到事务不能完成当前请求。ATT_BUSY也不会立即结束等待。

源码依据：App_Coordinator_Unicast_Receive 第810行。

问题27 终端重启后还能防重复吗

最后已提交远程修改的会话、事务、操作、槽位、结果及扫描模式随镜像保存。重启后相同事务返回原结果，不再次写。这个保护只有有限窗口，并不是保存所有历史事务或抵抗恶意重放的完整认证。

源码依据：App_Database_Execute 第320行；App_Database_Last_Scan_Mode 第85行。

问题28 CRC16有什么用 和RC522的CRC_A一样吗

ATT2和EEPROM镜像用初值0xFFFF、多项式0x1021的CRC16。RC522选卡使用初值0x6363的CRC_A路径。二者针对不同数据；CRC检测错误，不进行加密、卡片认证或保密。

源码依据：Comm_Protocol_CalcChecksum 第21行；Hal_Rc522_Crc_A 第139行。

问题29 为什么只保存8个用户和10条记录

每镜像32字节头部、8乘40字节用户、10乘16字节日志共512字节。AT24C08的1024字节要留双镜像，容量选择与可靠更新空间一起设计。扩大数量必须改布局及兼容版本，不能只改一个宏。

源码依据：App_Database_Execute 第320行；App_Database_Tick 第130行。

问题30 双镜像是否占了1024字节RAM

没有。终端RAM只保留一份512字节镜像，依次读两个EEPROM镜像记录有效性和代次，再装载选中的一份。32字节回读缓存和回复缓存另外存在。协调器统计才有两份小64字节RAM镜像。

源码依据：App_Database_Tick 第130行；App_Host_Store_Process 第170行。

问题31 为什么最后才写提交标记

目标镜像先标无效，再写所有内容并回读比较，最后写0xA5。中间掉电时新镜像不被信任，旧活动镜像仍在；标记和CRC都合格才可以选择新镜像。

源码依据：App_Database_Tick 第130行。

问题32 最后改标记不会破坏CRC吗

计算镜像CRC时暂时把提交标记和CRC字段归零，计算后恢复。校验同样使用该规则，所以最终标记变为0xA5不会使镜像CRC失配。

源码依据：App_Database_Image_Crc 第17行。

问题33 EEPROM损坏会自动清空吗

只有两个镜像都全FF的空白芯片建立RAM空库。非空且无有效镜像报CORRUPT，写失败报STORAGE并尝试装载旧库。确实要重建才执行DB INIT CONFIRM。

源码依据：App_Database_Tick 第130行；App_Database_Failed 第111行。

问题34 数据库Execute返回BUSY为什么可能是正常

调用前已确认没有其他写事务时，修改RAM后设置写状态并返回BUSY表示新提交已开始。后续Tick完成设置Done，再取最终回复。不能把这次BUSY当立即成功，也不能直接当作写失败。

源码依据：App_Terminal_Execute 第829行；App_Database_Execute 第320行。

问题35 写入失败后为何还要重读旧库

修改时RAM里的镜像已经变了。如果只报错而保留新RAM，查询会看见没有真正保存的数据。Failed安排装载旧有效镜像，使RAM恢复到持久化状态。

源码依据：App_Database_Failed 第111行。

问题36 第11条考勤会怎样 最新记录怎么找

下一写槽对10取模，第11条覆盖最早记录，已有数量保持10。最新第i条按next加10减1减i再取模。用户注销不清循环日志。

源码依据：App_Database_Execute 第320行。

问题37 12位学号为何日志只占6字节

用户表用12字节ASCII方便输入输出，日志把每两位数字打包成一个BCD字节，12位只用6字节。打印日志再拆高低半字节恢复字符串，不需要64位大整数或浮点。

源码依据：App_Database_Execute 第320行；App_Terminal_Show_Result 第587行。

问题38 AT24C08跨过255地址时如何读写

高地址位写入器件控制地址，0x50到0x53选择256字节块，低8位为块内字地址。读操作按块边界拆分，写操作还按16字节页边界拆分，避免页回绕覆盖。

源码依据：Hal_Eeprom_Device 第11行；Hal_Eeprom_Read 第33行。

问题39 EEPROM写后NACK一定是断线吗

不一定，内部写周期可能暂时不应答。驱动ACK轮询，20ms内NACK返回BUSY让以后事件继续，超时才返回TIMEOUT；其他总线错误按原状态返回。

源码依据：Hal_Eeprom_Process 第94行。

问题40 协调器统计为何与日志条数不同

统计累计协调器本机已确认的成功操作次数，16位饱和；日志保留两端最近10条考勤，注册和注销也不新增考勤日志。它们的统计对象和保留方式不同。

源码依据：App_Host_Store_Record 第121行；App_Host_Store_Count_Increment 第81行。

问题41 终端为什么不能独立设时间

协调器提供唯一时间源，防止两端各自设出不一致时间。终端入网、握手或手动SYNC请求协调器时间，写入自己的RTC后继续硬件走时。

源码依据：App_Terminal_Command_Process 第304行；App_Terminal_Hello 第794行。

问题42 协调器默认时间真的是上电当前时间吗

不是。有效RTC直接使用；无效时Seed使用固件编译时的DATE和TIME。编译到烧录、重启的间隔会产生偏差，演示前应串口设置准确北京时间。

源码依据：App_Coordinator_Peripherals_Process 第103行；Hal_Rtc_Seed_Build_Time 第143行。

问题43 CPU每秒加一才是RTC计时吗

RTC芯片自己计时，CPU秒任务只读取并显示。管理模式和结果倒计时由软件秒事件递减，它们与RTC硬件年月日计时是不同机制。

源码依据：Hal_Rtc_Read_Time 第52行；App_Ui_Second 第375行。

问题44 RTC无效还能保存考勤吗 为什么

允许保存，但valid为0，时间字段不能当准确打卡依据。这样保留操作发生过的记录；后续校时不擅自伪造此前真实发生时间。

源码依据：App_Terminal_Card_Process 第1017行；App_Database_Execute 第320行。

问题45 没有法拉电容 终端重启怎么办

不能依赖RTC掉电保时。网络和外设就绪后向协调器取时；若尚未取得有效时间，只显示未校时并记录valid为0。无线离线但RTC持续供电时仍可继续走时。

源码依据：App_Terminal_Peripherals_Process 第112行；App_Terminal_Network_Update 第173行。

问题46 I2C为什么需要上拉 在其他板上不加也能用

本驱动只拉低或释放线路，高电平靠总线上拉。其他板或OLED模块可能已有上拉，并不意味着总线可以没有上拉。供电和GND不可靠也会造成SCL超时，软件不能修复虚接。

源码依据：Hal_I2c_Init 第46行；Hal_I2c_Scl_Release 第15行。

问题47 I2C地址0x3C和0x78是什么关系

本工程接口使用7位地址0x3C，发送时左移一位再加读写位，因此写地址字节为0x78。不能把0x78再传给7位地址接口，让它再左移。RTC的0x32和EEPROM的0x50也遵循同样规则。

源码依据：Hal_I2c_Address 第176行；Hal_Oled_Init 第140行。

问题48 RC522读取4字节和7字节UID有何区别

通过CL1到CL3级联选卡，检查Cascade Tag和SAK级联位。带0x88的块贡献3字节UID，最后不级联块贡献4字节，形成4、7或10字节。每级检查BCC和选卡CRC。

源码依据：Hal_Rc522_Read_Uid 第256行。

问题49 能同时识别多张卡吗 卡片身份安全吗

当前只实现单卡选卡路径，碰撞会返回错误，没有完整多卡仲裁。UID绑定可被克隆，代码没有扇区认证，Zigbee配置SECURE为0；课程展示可以使用，不能直接声称是生产安全门禁。

源码依据：Hal_Rc522_Anticoll 第214行；Hal_Rc522_Read_Uid 第256行。

问题50 串口为什么分接收缓冲和命令解析缓存

HAL负责拼一条完整行和非法输入恢复，Common解析器复制到自己的固定命令体并分词，避免破坏HAL接收状态。arguments指针只在同步回调中有效，跨事件数据另行复制。

源码依据：Hal_Uart_Process_Rx 第169行；Device1_UART_ParseCMD 第191行。

问题51 为什么不在UART中断里执行查询或写EEPROM

中断只提供接收通知，完整解析、参数检查、数据库和无线请求在OSAL任务上下文做。否则长操作会延迟其他中断和协议栈，也难以处理整行输入和发送背压。

源码依据：Hal_Uart_Rx_Callback 第23行；SampleApp_SerialCMD 第227行。

问题52 串口缓冲256字节为何实际最多255

环形缓冲保留一个空位，用读写索引相等表示空，下一写索引等于读索引表示满。Flush只推进SDK真正接受的字节，写不进去时留到下一次事件。

源码依据：Hal_Uart_Queue_Byte 第50行；Hal_Uart_Flush 第142行。

问题53 LIST和LOG如何避免一次打印太多

维护列表类型、索引及等待标志，一次请求或输出一条，收到结果再推进。帮助和STATUS也按步骤输出；协调器等应答时暂停这些输出。串口空闲空间不足返回OUTPUT_BUSY而不是死等。

源码依据：App_Coordinator_Console_Process 第1126行；App_Terminal_Console_Process 第1184行。

问题54 OLED为什么不用1024字节整屏缓冲

CC2530 RAM有限，因此每次生成128字节页，只发最多31列。一页5次、8页40次事件。节省RAM同时让出任务调度，代价是画面分段更新，不能保证完整帧原子切换。

源码依据：App_Ui_Tick 第391行；Hal_Oled_Write_Chunk 第249行。

问题55 为什么全中文界面仍不能输入中文姓名

页面是固定文案的GBK点阵子集，姓名输入和数据库校验只接受ASCII。两者不是同一功能。任意中文姓名需要编码、字模覆盖、长度和存储布局等另外设计。

源码依据：App_Ui_Hanzi 第86行；App_Database_Identity 第289行。

问题56 LED和蜂鸣器是自己写的还是SDK提供的

LED直接使用SDK HalLedSet，按键GPIO读取使用HalKeyRead；项目只补消抖。蜂鸣器是独立hal_buzzer驱动，定时关闭事件在角色主文件调度，引脚全部来自board_config。

源码依据：Hal_Buzzer_Beep 第29行；Hal_Key_Poll 第14行。

问题57 程序能编译 是否说明所有硬件功能都验证完

不是。主机测试验证逻辑、协议字段、断电步骤和边界，IAR验证目标编译链接。真实射频、I2C波形、RTC掉电、OLED效果及实际烧录需实板证据。应区分软件结论和硬件结论。

源码依据：App_Database_Tick 第130行；SampleApp_ProcessEvent 第78行。

问题58 如果扩展到多终端或更多学生 先改哪里

多终端需地址映射、每端会话及在途事务、独立模式和超时，不是只去掉单终端地址检查。更多学生需重新安排镜像布局、容量、响应和UI，再处理旧数据库版本兼容及断电测试。

源码依据：App_Coordinator_Unicast_Receive 第810行；App_Database_Execute 第320行。

问题59 为什么不加入迟到早退判断

当前课程设计要求是注册、注销、刷卡记录和查询，没有班次、时间段或迟到规则。现在记录时间和有效性，为后续规则提供数据，但不把未实现的规则说成已有功能。

源码依据：App_Database_Execute 第320行。

问题60 普通版代码里为什么看不到网站上传

当前普通版禁用ESP，MQTT在另一个独立副本里。扩展有一份ESP驱动C和一份业务C，并由协调器独立事件调度。网站演示样例也不能作为真实硬件上传证据。

源码依据：SampleApp_Init 第47行。

扩展版本和后续修改位置

当前普通版的board_config中BOARD_ESP_ENABLED为0，不含ESP业务模块。另一项目RC522大作业_ESP12F_MQTT有hal/hal_esp12f.c和app/app_esp12f.c，分别负责AT串口驱动及MQTT业务，主文件只调度接口。其网站后端在website/server.py，前端在website/static，普通版不能仅通过#HELP就启用云端。

若答辩展示扩展，另读扩展README及代码，区分实际MQTT设备数据与网页演示样例；缺Wi-Fi配置或未确认AT固件时，不声称真实刷卡已经上传。这些扩展不计入本册普通版204个函数清单。

增加新业务时，角色主文件加入明确事件，业务放app，外设操作放hal，参数放已有配置头；不要重新创建通用任务层。增加用户或日志容量，先重算512字节镜像布局和版本兼容；修改页大小，先确认实际EEPROM型号；增加汉字，更新字体生成脚本；改角色、PAN或信道，改对应IAR配置及f8w配置后双端重新验证。

全部文件和函数阅读索引

在用文件清单

文件	行数	职责
Source/Coordinator.c	233	协调器任务调度：初始化、AF/ZDO消息分发和逐项OSAL事件。
Source/EndDevice.c	220	终端任务调度：初始化、AF消息分发和逐项OSAL事件。
Source/SampleApp.h	104	两角色共用的端点、Cluster、事件位和周期配置。
app/Common/comm_broadcast.c	123	直接恢复旧 Common 广播接口，AF 消息仍按 Cluster 分发。
app/Common/comm_broadcast.h	55	同名实现的公开类型、常量和接口声明。
app/Common/comm_protocol.c	141	旧 Common 协议入口，按现有 ATT2 字段打包、校验、解包。
app/Common/comm_protocol.h	96	同名实现的公开类型、常量和接口声明。
app/Common/comm_unicast.c	125	直接恢复旧 Common 单播接口，AF 发送与接收回调。
app/Common/comm_unicast.h	57	同名实现的公开类型、常量和接口声明。
app/Common/device1_uart.c	280	旧 Device1_UART 串口模块：找命令、判断类型、拆参数、回调。
app/Common/device1_uart.h	62	同名实现的公开类型、常量和接口声明。
app/app_coordinator.c	1270	协调器业务：串口管理、模式同步、远程刷卡、应答和时间源。
app/app_coordinator.h	21	同名实现的公开类型、常量和接口声明。
app/app_database.c	565	终端AT24C08权威数据库：名单、循环日志、双镜像与重复事务保护。
app/app_database.h	20	同名实现的公开类型、常量和接口声明。
app/app_font.h	252	Noto Sans SC生成的GBK 16x16点阵与中文文案。
app/app_host_store.c	303	协调器AT24C02本机统计：成功应答后计数，双镜像异步保存。
app/app_host_store.h	18	同名实现的公开类型、常量和接口声明。
app/app_terminal.c	1307	终端业务：本地刷卡、名单管理、异步提交、远程响应和考勤上报。
app/app_terminal.h	21	同名实现的公开类型、常量和接口声明。

文件	行数	职责
app/app_ui.c	427	中文OLED业务页面与128字节页缓冲，按事件增量刷新。
app/app_ui.h	11	同名实现的公开类型、常量和接口声明。
hal/board/board_config.h	75	自制板引脚、电气默认状态和ESP禁用配置。
hal/hal_board_key.c	36	板上按键消抖；原SDK hal_key负责引脚读取。
hal/hal_board_key.h	9	同名实现的公开类型、常量和接口声明。
hal/hal_buzzer.c	44	P0.0蜂鸣器独立驱动，OSAL单次定时事件关闭输出。
hal/hal_buzzer.h	11	同名实现的公开类型、常量和接口声明。
hal/hal_console_uart.c	224	UART0唯一拥有者：38400、收发缓存、分行、背压和OSAL接收事件。
hal/hal_console_uart.h	18	同名实现的公开类型、常量和接口声明。
hal/hal_delay.c	26	原项目的短延时接口，不承担业务调度。
hal/hal_delay.h	9	同名实现的公开类型、常量和接口声明。
hal/hal_eeprom.c	141	AT24C02/08的块地址、分页读写和有界ACK轮询。
hal/hal_eeprom.h	18	同名实现的公开类型、常量和接口声明。
hal/hal_i2c.c	258	P1.2/P1.1软件开漏I2C、总线释放和应答时序。
hal/hal_i2c.h	20	同名实现的公开类型、常量和接口声明。
hal/hal_oled.c	267	SSD1306地址探测、命令、ASCII字模和分段数据写入。
hal/hal_oled.h	15	同名实现的公开类型、常量和接口声明。
hal/hal_rc522.c	296	软件SPI、MFRC522选卡和4/7/10字节UID读取。
hal/hal_rc522.h	21	同名实现的公开类型、常量和接口声明。
hal/hal_rtc.c	198	RX8010日期校验、有效标记、读写时钟；编译时间仅供协调器初始化。
hal/hal_rtc.h	16	同名实现的公开类型、常量和接口声明。

协调器任务文件

文件：Source/Coordinator.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
SampleApp_Init 第47到76行 `void SampleApp_Init(uint8 task_id)`	注册 AF 端点和通信回调，启动本地周期事件；ZDO 自动建网由原协议栈负责。 由SDK任务注册与调度调用。
SampleApp_ProcessEvent 第78到196行 `uint16 SampleApp_ProcessEvent(uint8 task_id, uint16 events)`	按旧项目 OSAL 写法处理一个事件位并返回其余位；消息事件先取完队列并释放每条消息。 由SDK任务注册与调度调用。
SampleApp_MessageMSGCB 第198到215行 `static void SampleApp_MessageMSGCB(afIncomingMSGPacket_t *packet)`	按端点 Cluster 分发 AF 消息到旧项目单播或广播模块。 被引用：SampleApp_ProcessEvent。
Coordinator_Zdo_Message_Process 第217到224行 `static void Coordinator_Zdo_Message_Process(zdoIncomingMsg_t *message)`	接收旧项目的设备入网公告；仅记录事件，终端地址仍以 ATT2 HELLO 为准。 被引用：SampleApp_ProcessEvent。
SampleApp_SerialCMD 第227到233行 `static void SampleApp_SerialCMD(char *line)`	原式串口入口：完整行交给 Device1_UART_ProcessCMD，解析失败返回语法提示。 被引用：SampleApp_Init。

终端任务文件

文件：Source/EndDevice.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
SampleApp_Init 第46到74行 `void SampleApp_Init(uint8 task_id)`	注册 AF 端点和通信回调，启动本地周期事件；ZDO 自动建网由原协议栈负责。 由SDK任务注册与调度调用。
SampleApp_ProcessEvent 第76到192行 `uint16 SampleApp_ProcessEvent(uint8 task_id, uint16 events)`	按旧项目 OSAL 写法处理一个事件位并返回其余位；消息事件先取完队列并释放每条消息。 由SDK任务注册与调度调用。
SampleApp_MessageMSGCB 第194到211行 `static void SampleApp_MessageMSGCB(afIncomingMSGPacket_t *packet)`	按端点 Cluster 分发 AF 消息到旧项目单播或广播模块。 被引用：SampleApp_ProcessEvent。
SampleApp_SerialCMD 第214到220行 `static void SampleApp_SerialCMD(char *line)`	原式串口入口：完整行交给 Device1_UART_ProcessCMD，解析失败返回语法提示。 被引用：SampleApp_Init。

协调器业务模块

文件：app/app_coordinator.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
App_Coordinator_Init 第84到101行 `void App_Coordinator_Init(uint8 task_id)`	初始化角色业务状态、LED、按键和蜂鸣器；串口、通信、端点与定时事件由角色主文件注册。 被引用：SampleApp_Init。
App_Coordinator_Peripherals_Process 第103到155行 `uint8 App_Coordinator_Peripherals_Process(void)`	按调用次数依次初始化外设；准备完成返回1，由角色任务启动刷卡和存储事件。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Network_Update 第157到168行 `void App_Coordinator_Network_Update(uint8 joined)`	接收ZDO状态变化；更新联网状态，终端就绪时立即发送HELLO取协调器时间。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Database_Process 第170到173行 `void App_Coordinator_Database_Process(void)`	推进本角色EEPROM服务；终端完成提交后才应答、显示结果并通知协调器。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Display_Process 第175到184行 `void App_Coordinator_Display_Process(void)`	读取角色业务状态并推进中文界面的分段刷新。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Retry_Process 第186到210行 `void App_Coordinator_Retry_Process(void)`	处理两秒无线重试；终端通知超时不会撤销已保存考勤或阻塞本机操作。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Key_Process 第213到234行 `void App_Coordinator_Key_Process(uint8 shift, uint8 keys)`	把消抖后的两个实体按键转换为注册、注销或取消模式。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Command_Process 第236到515行 `void App_Coordinator_Command_Process(uart_cmd_t *command)`	检查就绪、TX空间和命令参数；TIME/DB分组复用原业务动作。 被引用：SampleApp_Init。
App_Coordinator_Result_Text 第518到545行 `static const char *App_Coordinator_Result_Text(uint8 status)`	把 ATT2 结果码转换成便于串口阅读的文字。 被引用：App_Coordinator_Show_Result。
App_Coordinator_Send 第547到554行 `static void App_Coordinator_Send(uint16 destination, uint8 opcode, uint16 session, uint16 transaction, const uint8 *payload, uint8 payload_length)`	封装 ATT2 帧并通过旧项目 Comm_Unicast 模块发送。 被引用：App_Coordinator_Unicast_Receive。
App_Coordinator_Hex 第556到566行 `static void App_Coordinator_Hex(const uint8 *uid, uint8 uid_length, char *text)`	把 UID 字节编码为串口使用的十六进制字符串。 被引用：App_Coordinator_Card_Process、App_Coordinator_Command_Process、App_Coordinator_Show_Result。
App_Coordinator_Show_Result 第568到709行 `static void App_Coordinator_Show_Result(uint8 opcode, const uint8 *payload, uint8 payload_length)`	统一更新串口与 OLED 的业务结果提示。 被引用：App_Coordinator_Request、App_Coordinator_Retry_Process、App_Coordinator_Unicast_Receive。

函数及定位	用途及引用关系
App_Coordinator_Set_Mode 第711到734行 `static void App_Coordinator_Set_Mode(uint8 new_mode, const uint8 *identity)`	保存注册或注销模式及有效期，并同步界面状态。 被引用：App_Coordinator_Begin_Mode、App_Coordinator_Card_Process、App_Coordinator_Second_Process。
App_Coordinator_Time_Payload 第736到744行 `static void App_Coordinator_Time_Payload(uint8 *payload)`	把当前 RTC 日期时间填入 ATT2 时间负载。 被引用：App_Coordinator_Card_Process、App_Coordinator_Command_Process、App_Coordinator_Second_Process。
App_Coordinator_Set_Time 第746到778行 `static uint8 App_Coordinator_Set_Time(const uint8 *payload)`	校验无线时间负载并写入本机 RTC。 被引用：App_Coordinator_Command_Process。
App_Coordinator_Request 第780到807行 `static void App_Coordinator_Request(uint8 opcode, const uint8 *payload, uint8 payload_length)`	只有终端在线且当前事务空闲时才发送命令，并保存帧供超时重发。 被引用：App_Coordinator_Begin_Mode、App_Coordinator_Card_Process、App_Coordinator_Command_Process等。
App_Coordinator_Unicast_Receive 第810到923行 `void App_Coordinator_Unicast_Receive(uint16 source, uint8 *raw, uint8 frame_length)`	校验 ATT2 来源、会话和事务；HELLO 确认终端，事件回 ACK，请求应答停止重发。 被引用：SampleApp_Init。
App_Coordinator_Card_Process 第926到968行 `void App_Coordinator_Card_Process(void)`	轮询 RC522；同一张卡离开读卡区后才允许再次触发业务。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Network_Status 第970到985行 `static void App_Coordinator_Network_Status(uint8 line)`	按line分条输出网络和外设；由CONSOLE步骤推进，避免连续占满TX。 被引用：App_Coordinator_Console_Process。
App_Coordinator_Begin_Mode 第987到1030行 `static void App_Coordinator_Begin_Mode(uint8 new_mode, const uint8 *identity)`	由按键或串口切换刷卡模式，并处理忙状态和倒计时。 被引用：App_Coordinator_Command_Process、App_Coordinator_Key_Process、App_Coordinator_Second_Process。
App_Coordinator_Parse_Index 第1032到1055行 `static uint8 App_Coordinator_Parse_Index(const char *text, uint8 limit, uint8 *output_value)`	解析用户或日志索引，拒绝非数字及越界输入。 被引用：App_Coordinator_Command_Process。
App_Coordinator_Parse_Identity 第1057到1080行 `static uint8 App_Coordinator_Parse_Identity(char *student_id, char *student_name, uint8 *identity)`	校验十二位学号和姓名，将注册资料填入协议负载。 被引用：App_Coordinator_Command_Process。
App_Coordinator_Hex_Digit 第1082到1097行 `static uint8 App_Coordinator_Hex_Digit(char character)`	解析单个十六进制字符；非法字符返回错误标记。 被引用：App_Coordinator_Parse_Uid。
App_Coordinator_Parse_Uid 第1099到1124行 `static uint8 App_Coordinator_Parse_Uid(const char *text, uint8 *uid)`	把串口 UID 文本解析为 4、7 或 10 字节卡号。 被引用：App_Coordinator_Command_Process。

函数及定位	用途及引用关系
App_Coordinator_Console_Process 第1126到1209行 `void App_Coordinator_Console_Process(void)`	按发送缓冲空间逐步输出帮助、用户列表或考勤日志。 被引用：SampleApp_ProcessEvent。
App_Coordinator_Second_Process 第1212到1270行 `void App_Coordinator_Second_Process(void)`	每秒读取 RTC、刷新在线状态，并在条件满足时向终端同步时间。 被引用：SampleApp_ProcessEvent。

终端业务模块

文件：app/app_terminal.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
App_Terminal_Init 第93到110行 `void App_Terminal_Init(uint8 task_id)`	初始化角色业务状态、LED、按键和蜂鸣器；串口、通信、端点与定时事件由角色主文件注册。 被引用：SampleApp_Init。
App_Terminal_Peripherals_Process 第112到171行 `uint8 App_Terminal_Peripherals_Process(void)`	按调用次数依次初始化外设；准备完成返回1，由角色任务启动刷卡和存储事件。 被引用：SampleApp_ProcessEvent。
App_Terminal_Network_Update 第173到194行 `void App_Terminal_Network_Update(uint8 joined)`	接收ZDO状态变化；更新联网状态，终端就绪时立即发送HELLO取协调器时间。 被引用：SampleApp_ProcessEvent。
App_Terminal_Database_Process 第196到247行 `void App_Terminal_Database_Process(void)`	推进本角色EEPROM服务；终端完成提交后才应答、显示结果并通知协调器。 被引用：SampleApp_ProcessEvent。
App_Terminal_Display_Process 第249到258行 `void App_Terminal_Display_Process(void)`	读取角色业务状态并推进中文界面的分段刷新。 被引用：SampleApp_ProcessEvent。
App_Terminal_Retry_Process 第260到278行 `void App_Terminal_Retry_Process(void)`	处理两秒无线重试；终端通知超时不会撤销已保存考勤或阻塞本机操作。 被引用：SampleApp_ProcessEvent。
App_Terminal_Key_Process 第281到302行 `void App_Terminal_Key_Process(uint8 shift, uint8 keys)`	把消抖后的两个实体按键转换为注册、注销或取消模式。 被引用：SampleApp_ProcessEvent。
App_Terminal_Command_Process 第304到534行 `void App_Terminal_Command_Process(uart_cmd_t *command)`	执行本地查询及管理；独立校时被拒绝，TIME SYNC向协调器取时。 被引用：SampleApp_Init。
App_Terminal_Result_Text 第537到564行 `static const char *App_Terminal_Result_Text(uint8 status)`	把 ATT2 结果码转换成便于串口阅读的文字。 被引用：App_Terminal_Show_Result。

函数及定位	用途及引用关系
App_Terminal_Send 第566到573行 `static void App_Terminal_Send(uint16 destination, uint8 opcode, uint16 session, uint16 transaction, const uint8 *payload, uint8 payload_length)`	封装 ATT2 帧并通过旧项目 Comm_Unicast 模块发送。 被引用：App_Terminal_Database_Process、App_Terminal_Execute、App_Terminal_Hello等。
App_Terminal_Hex 第575到585行 `static void App_Terminal_Hex(const uint8 *uid, uint8 uid_length, char *text)`	把 UID 字节编码为串口使用的十六进制字符串。 被引用：App_Terminal_Card_Process、App_Terminal_Command_Process、App_Terminal_Show_Result。
App_Terminal_Show_Result 第587到723行 `static void App_Terminal_Show_Result(uint8 opcode, const uint8 *payload, uint8 payload_length)`	统一更新串口与 OLED 的业务结果提示。 被引用：App_Terminal_Database_Process、App_Terminal_Execute。
App_Terminal_Set_Mode 第725到748行 `static void App_Terminal_Set_Mode(uint8 new_mode, const uint8 *identity)`	保存注册或注销模式及有效期，并同步界面状态。 被引用：App_Terminal_Begin_Mode、App_Terminal_Card_Process、App_Terminal_Second_Process等。
App_Terminal_Time_Payload 第750到758行 `static void App_Terminal_Time_Payload(uint8 *payload)`	把当前 RTC 日期时间填入 ATT2 时间负载。 被引用：App_Terminal_Card_Process。
App_Terminal_Set_Time 第760到792行 `static uint8 App_Terminal_Set_Time(const uint8 *payload)`	校验无线时间负载并写入本机 RTC。 被引用：App_Terminal_Unicast_Receive。
App_Terminal_Hello 第794到805行 `static void App_Terminal_Hello(uint8 time_sync_request)`	发7字节数据库及模式概要，手动校时请求仅改变载荷标记，不破坏有效RTC。 被引用：App_Terminal_Begin_Mode、App_Terminal_Command_Process、App_Terminal_Database_Process等。
App_Terminal_Complete_Reply 第807到827行 `static uint8 App_Terminal_Complete_Reply(uint8 opcode, uint8 *response)`	把数据库完成后的状态、槽位和代数写入应答负载。 被引用：App_Terminal_Database_Process、App_Terminal_Execute。
App_Terminal_Execute 第829到887行 `static void App_Terminal_Execute(uint8 opcode, const uint8 *payload, uint8 payload_length, uint16 request_session, uint16 request_transaction, uint8 remote)`	执行本地或远程考勤/管理命令；持久化操作等待数据库异步提交。 被引用：App_Terminal_Request、App_Terminal_Unicast_Receive。
App_Terminal_Request 第889到893行 `static void App_Terminal_Request(uint8 opcode, const uint8 *payload, uint8 payload_length)`	执行本机串口或刷卡命令，统一经过终端数据库业务入口。 被引用：App_Terminal_Card_Process、App_Terminal_Command_Process、App_Terminal_Console_Process。
App_Terminal_Unicast_Receive 第896到1014行 `void App_Terminal_Unicast_Receive(uint16 source, uint8 *raw, uint8 frame_length)`	只接收协调器短地址 0000 的 ATT2 帧，过滤重复事务并发送回复。 被引用：SampleApp_Init。

函数及定位	用途及引用关系
App_Terminal_Card_Process 第1017到1058行 `void App_Terminal_Card_Process(void)`	轮询 RC522；同一张卡离开读卡区后才允许再次触发业务。 被引用：SampleApp_ProcessEvent。
App_Terminal_Network_Status 第1060到1075行 `static void App_Terminal_Network_Status(uint8 line)`	按line分条输出终端网络和外设；串口STATUS设置输出步骤。 被引用：App_Terminal_Console_Process。
App_Terminal_Begin_Mode 第1077到1093行 `static void App_Terminal_Begin_Mode(uint8 new_mode, const uint8 *identity)`	由按键或串口切换刷卡模式，并处理忙状态和倒计时。 被引用：App_Terminal_Command_Process、App_Terminal_Key_Process。
App_Terminal_Number 第1095到1118行 `static uint8 App_Terminal_Number(const char *text, uint8 limit, uint8 *output_value)`	解析用户或日志索引，拒绝非数字及越界输入。 被引用：App_Terminal_Command_Process。
App_Terminal_Parse_Identity 第1120到1143行 `static uint8 App_Terminal_Parse_Identity(char *student_id, char *student_name, uint8 *identity)`	校验十二位学号和姓名，将注册资料填入协议负载。 被引用：App_Terminal_Command_Process。
App_Terminal_Hex_Digit 第1145到1160行 `static uint8 App_Terminal_Hex_Digit(char character)`	解析单个十六进制字符；非法字符返回错误标记。 被引用：App_Terminal_Parse_Uid。
App_Terminal_Parse_Uid 第1162到1182行 `static uint8 App_Terminal_Parse_Uid(const char *text, uint8 *uid)`	把串口 UID 文本解析为 4、7 或 10 字节卡号。 被引用：App_Terminal_Command_Process。
App_Terminal_Console_Process 第1184到1262行 `void App_Terminal_Console_Process(void)`	按发送缓冲空间逐步输出帮助、用户列表或考勤日志。 被引用：SampleApp_ProcessEvent。
App_Terminal_Second_Process 第1265到1307行 `void App_Terminal_Second_Process(void)`	每秒读取 RTC、维护模式倒计时和在线状态，并周期发送 HELLO。 被引用：SampleApp_ProcessEvent。

单播通信模块

文件：app/Common/comm_unicast.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Comm_Unicast_Init 第32到42行 `void Comm_Unicast_Init(uint8 task_id)`	重置AF事务号和接收回调，配置16位单播目标地址及端点。 被引用：SampleApp_Init。
Comm_Unicast_Send 第54到87行 `uint8 Comm_Unicast_Send(uint16 dst_addr, uint8 *data, uint8 len)`	检查数据参数，调用AF_DataRequest；返回的是发送接受状态，不是业务提交结果。 被引用：App_Coordinator_Request、App_Coordinator_Retry_Process、App_Coordinator_Send等。

函数及定位	用途及引用关系
Comm_Unicast_RegisterCallback 第96到99行 `void Comm_Unicast_RegisterCallback(Comm_Unicast_RecvCallback callback)`	登记同步接收回调；不创建新的应用任务。 被引用：SampleApp_Init。
Comm_Unicast_ProcessMSG 第109到125行 `void Comm_Unicast_ProcessMSG(afIncomingMSGPacket_t *pkt)`	提取源短地址和AF数据，存在回调时同步分发。 被引用：SampleApp_MessageMSGCB。

广播接口模块

文件：app/Common/comm_broadcast.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Comm_Broadcast_Init 第33到43行 `void Comm_Broadcast_Init(uint8 task_id)`	重置广播事务和回调，配置广播目标及端点；当前核心业务未使用广播。 被引用：SampleApp_Init。
Comm_Broadcast_Send 第55到85行 `uint8 Comm_Broadcast_Send(uint8 *data, uint8 len)`	按广播Cluster调用AF发送；不替代单播数据库事务。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。
Comm_Broadcast_RegisterCallback 第94到97行 `void Comm_Broadcast_RegisterCallback(Comm_Broadcast_RecvCallback callback)`	注册广播业务回调；当前角色主文件没有启用广播业务回调。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。
Comm_Broadcast_ProcessMSG 第107到123行 `void Comm_Broadcast_ProcessMSG(afIncomingMSGPacket_t *pkt)`	有回调时提取并分发广播数据，否则不执行额外考勤业务。 被引用：SampleApp_MessageMSGCB。

ATT2编解码模块

文件：app/Common/comm_protocol.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Comm_Protocol_ReadUint16 第9到12行 `uint16 Comm_Protocol_ReadUint16(const uint8 *data)`	从两个字节读取小端16位数；调用者保证缓冲长度。 被引用：App_Coordinator_Show_Result、App_Coordinator_Unicast_Receive、App_Database_Execute等。
Comm_Protocol_WriteUint16 第14到18行 `void Comm_Protocol_WriteUint16(uint8 *data, uint16 value)`	把16位数写为低字节在前；用于会话、事务、代次及CRC。 被引用：App_Database_Empty、App_Database_Execute、App_Database_Image_Crc等。

函数及定位	用途及引用关系
Comm_Protocol_CalcChecksum 第21到43行 `uint16 Comm_Protocol_CalcChecksum(const uint8 *data, uint16 len)`	CRC16初值FFFF、多项式1021，校验通信帧和数据库镜像。 被引用：App_Database_Image_Crc、App_Host_Store_Image_Crc、Comm_Protocol_PackFrame等。
Comm_Protocol_PackFrame 第46到86行 `uint8 Comm_Protocol_PackFrame(uint8 *buf, uint8 buf_len, uint8 opcode, uint16 session, uint16 transaction, const uint8 *data, uint8 data_len)`	检查容量和载荷，填ATT2头、复制数据并追加CRC，失败返回0长度。 被引用：App_Coordinator_Request、App_Coordinator_Send、App_Terminal_Database_Process等。
Comm_Protocol_ValidateFrame 第89到117行 `uint8 Comm_Protocol_ValidateFrame(const uint8 *raw, uint8 raw_len)`	检查空指针、帧长、头、版本、载荷长度和CRC，返回COMM域错误。 被引用：Comm_Protocol_UnpackFrame。
Comm_Protocol_UnpackFrame 第120到141行 `uint8 Comm_Protocol_UnpackFrame(comm_frame_t *frame, const uint8 *raw, uint8 raw_len)`	验证后公开opcode及借用的数据指针，只在原消息有效期内使用。 被引用：App_Coordinator_Unicast_Receive、App_Terminal_Unicast_Receive。

旧式串口解析模块

文件：app/Common/device1_uart.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Device1_UART_Init 第17到21行 `void Device1_UART_Init(uint8 task_id)`	仅重置解析回调，不重复初始化串口硬件。 被引用：SampleApp_Init。
Device1_UART_SendData 第23到27行 `void Device1_UART_SendData(uint8 *data, uint8 len)`	旧式发送入口，按给定长度进入HAL固定TX缓冲。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。
Device1_UART_SendString 第29到32行 `void Device1_UART_SendString(uint8 *str, uint8 len)`	旧式字符串发送入口，同样按长度转给HAL。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。
UART_FindCMD 第35到80行 `static uint8 UART_FindCMD(uint8 *raw, uint8 len, uint8 *start, uint8 *end)`	找首个井号、确认尾部并去行尾空白，拒绝空命令和混入第二条命令。 被引用：Device1_UART_ParseCMD。
UART_GetCMDType 第83到182行 `static uart_cmd_type_t UART_GetCMDType(uart_cmd_t *cmd)`	分词后识别六类公开命令，归一DB动作及LOG CSV参数位置。 被引用：Device1_UART_ParseCMD。
UART_IsSeparator 第185到188行 `static uint8 UART_IsSeparator(uint8 value)`	识别空格、TAB和冒号为参数分隔符。 被引用：Device1_UART_ParseCMD。
Device1_UART_ParseCMD 第191到259行 `uint8 Device1_UART_ParseCMD(uint8 *raw, uint8 len, uart_cmd_t *cmd)`	复制命令到固定80字节体，最多分5项，参数指针仅当前同步回调有效。 被引用：Device1_UART_ProcessCMD。

函数及定位	用途及引用关系
Device1_UART_ProcessCMD 第262到275行 `uint8 Device1_UART_ProcessCMD(uint8 *raw, uint8 len)`	解析完整命令并同步调用已注册的业务回调。 被引用：SampleApp_SerialCMD。
Device1_UART_RegisterCallback 第277到280行 `void Device1_UART_RegisterCallback(Device1_UARTRecvCallback callback)`	保存当前角色的业务处理函数指针。 被引用：SampleApp_Init。

终端数据库模块

文件：app/app_database.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
App_Database_Image_Crc 第17到27行 `static uint16 App_Database_Image_Crc(void)`	计算 512 字节镜像 CRC；计算时临时清除提交标记和 CRC 字段。 被引用：App_Database_Execute、App_Database_Image_Valid。
App_Database_Image_Valid 第28到49行 `static uint8 App_Database_Image_Valid(void)`	校验镜像头、版本、提交标记、循环日志范围、用户槽和 CRC。 被引用：App_Database_Tick。
App_Database_Empty 第50到56行 `static void App_Database_Empty(void)`	在 RAM 中建立空数据库镜像；此时尚未写入 EEPROM。 被引用：App_Database_Execute、App_Database_Failed、App_Database_Tick。
App_Database_Init 第57到67行 `void App_Database_Init(void)`	配置终端 AT24C08，并启动两个 512 字节镜像的逐块扫描。 被引用：App_Terminal_Peripherals_Process。
App_Database_Status 第68到71行 `uint8 App_Database_Status(void)`	返回数据库状态；初始化和提交期间统一报告忙。 被引用：App_Database_Count、App_Database_User、App_Terminal_Database_Process等。
App_Database_Generation 第72到75行 `uint16 App_Database_Generation(void)`	读取当前镜像代数，用于选择更新的一份和应答。 被引用：App_Database_Execute、App_Database_Tick、App_Terminal_Hello等。
App_Database_User 第76到83行 `const uint8 *App_Database_User(uint8 slot)`	按槽位返回已注册用户记录；状态异常或空槽返回空指针。 被引用：App_Terminal_Complete_Reply、App_Terminal_Show_Result。
App_Database_Last_Scan_Mode 第85到88行 `uint8 App_Database_Last_Scan_Mode(void)`	读取最后一次持久化远程刷卡的实际模式，重启后重发仍返回原结果类型。 被引用：App_Terminal_Execute。
App_Database_Request_Fresh 第90到94行 `uint8 App_Database_Request_Fresh(uint16 session, uint16 tx)`	用会话和 16 位事务号判断远程刷卡请求是否比已持久化请求更新。 被引用：App_Terminal_Execute、App_Terminal_Unicast_Receive。

函数及定位	用途及引用关系
App_Database_Count 第95到110行 `uint8 App_Database_Count(void)`	统计八个用户槽中已注册的卡片数量。 被引用：App_Database_Execute、App_Terminal_Database_Process、App_Terminal_Hello等。
App_Database_Failed 第111到129行 `static void App_Database_Failed(void)`	写入失败后重新装入上一个已提交镜像，避免把 RAM 未落盘结果当成功。 被引用：App_Database_Tick。
App_Database_Tick 第130到275行 `void App_Database_Tick(void)`	每次推进一次镜像读取或写入状态机；先使目标镜像失效，再写入、回读校验并提交。 被引用：App_Terminal_Database_Process、App_Terminal_Peripherals_Process。
App_Database_Find 第276到288行 `static uint8 App_Database_Find(const uint8 *uid, uint8 n)`	在八个用户槽中查找相同长度和内容的 UID。 被引用：App_Database_Execute。
App_Database_Identity 第289到319行 `static uint8 App_Database_Identity(const uint8 *id, const uint8 *name, uint8 except)`	校验十二位学号与姓名，并阻止不同卡使用重复学号。 被引用：App_Database_Execute。
App_Database_Execute 第320到551行 `uint8 App_Database_Execute(uint8 op, const uint8 *p, uint8 n, uint16 session, uint16 tx, uint8 remote)`	处理查询、注册、注销、考勤、修改与格式化；修改返回忙并由 DB_Tick 异步落盘。 被引用：App_Terminal_Execute。
App_Database_Take_Done 第552到557行 `uint8 App_Database_Take_Done(void)`	消费一次异步提交完成标记，避免重复发送结果。 被引用：App_Terminal_Database_Process。
App_Database_Reply 第558到561行 `const uint8 *App_Database_Reply(void)`	返回最后一次操作的回复缓存；由调用方按 DB_ReplySize 读取。 被引用：App_Terminal_Complete_Reply、App_Terminal_Execute。
App_Database_Reply_Size 第562到565行 `uint8 App_Database_Reply_Size(void)`	返回回复缓存当前有效字节数。 被引用：App_Terminal_Complete_Reply、App_Terminal_Execute。

协调器统计模块

文件：app/app_host_store.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
App_Host_Store_Image_Crc 第25到38行 `static uint16 App_Host_Store_Image_Crc(uint8 *image)`	暂时清除提交标记和CRC字段计算镜像校验值，然后恢复RAM内容。 被引用：App_Host_Store_Image_Valid、App_Host_Store_Save_Start。

函数及定位	用途及引用关系
App_Host_Store_Image_Valid 第40到44行 `static uint8 App_Host_Store_Image_Valid(uint8 *image)`	检查统计镜像魔数、版本、提交标记和CRC，拒绝未完整提交的镜像。 被引用：App_Host_Store_Process。
App_Host_Store_Is_Blank 第46到61行 `static uint8 App_Host_Store_Is_Blank(void)`	检查两个统计镜像是否全为FF；只有全空芯片可以自动建立空统计。 被引用：App_Host_Store_Process。
App_Host_Store_Image_Empty 第63到68行 `static void App_Host_Store_Image_Empty(uint8 *image)`	在RAM准备空统计镜像，不立即写EEPROM。 被引用：App_Host_Store_Process、App_Host_Store_Reset。
App_Host_Store_Save_Start 第70到79行 `static void App_Host_Store_Save_Start(void)`	更新目标镜像代次与CRC，启动失效、写入、回读和最后提交过程。 被引用：App_Host_Store_Process、App_Host_Store_Record、App_Host_Store_Reset。
App_Host_Store_Count_Increment 第81到90行 `static void App_Host_Store_Count_Increment(uint8 *value)`	增加16位累计计数；到65535后饱和，避免回绕误显示为零。 被引用：App_Host_Store_Process、App_Host_Store_Record。
App_Host_Store_Init 第92到100行 `void App_Host_Store_Init(void)`	配置协调器AT24C02并安排两个64字节镜像的逐步读取。 被引用：App_Coordinator_Peripherals_Process。
App_Host_Store_Status 第102到105行 `uint8 App_Host_Store_Status(void)`	报告统计存储状态；初始化或提交期间返回忙。 被引用：App_Coordinator_Card_Process、App_Coordinator_Command_Process。
App_Host_Store_Count 第107到114行 `uint16 App_Host_Store_Count(uint8 mode)`	按考勤、注册、注销模式读取已经提交的本机计数。 被引用：App_Coordinator_Command_Process。
App_Host_Store_Boot_Count 第116到119行 `uint16 App_Host_Store_Boot_Count(void)`	读取已经提交的启动次数，不把尚未提交的数据当成成功。 被引用：App_Coordinator_Command_Process。
App_Host_Store_Record 第121到156行 `uint8 App_Host_Store_Record(uint8 mode, const uint8 *uid, uint8 uid_length, const rtc_time_t *time, uint8 time_status)`	复制活动镜像，记录本机成功扫描的计数、UID和原始时间；ATT_OK仅表示接受异步请求。 被引用：App_Coordinator_Unicast_Receive。
App_Host_Store_Reset 第158到168行 `uint8 App_Host_Store_Reset(void)`	在用户明确确认后建立空统计镜像，保留原镜像直到新提交成功。 被引用：App_Coordinator_Command_Process。
App_Host_Store_Process 第170到303行 `void App_Host_Store_Process(void)`	每10ms推进一次读取或提交；写入失败保留上一份活动统计。 被引用：App_Coordinator_Database_Process。

中文界面业务模块

文件：app/app_ui.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
App_Ui_Copy 第16到25行 `static void App_Ui_Copy(char *target, const char *source)`	复制一条界面文案并补字符串结束符。 被引用：App_Ui_Message、App_Ui_Render。
App_Ui_Chinese 第28到77行 `static const char *App_Ui_Chinese(const char *text)`	将现有英文业务状态映射为 OLED 中文文案，保持串口与无线协议不变。 被引用：App_Ui_Message。
App_Ui_Pair 第79到83行 `static void App_Ui_Pair(char *target, uint8 value)`	将 0 到 99 转换为两位十进制字符。 被引用：App_Ui_Render。
App_Ui_Hanzi 第86到108行 `static const att_hanzi_t *App_Ui_Hanzi(uint16 code)`	按 GBK 编码二分查找 16x16 汉字点阵。 被引用：App_Ui_Text。
App_Ui_Text 第111到157行 `static void App_Ui_Text(uint8 x, const char *string, uint8 half)`	在当前页绘制汉字的上半或下半字形，并兼容 ASCII 数字。 被引用：App_Ui_Center、App_Ui_Render。
App_Ui_Width 第159到176行 `static uint8 App_Ui_Width(const char *string)`	计算中英混排内容的像素宽度，供居中显示使用。 被引用：App_Ui_Center。
App_Ui_Center 第178到182行 `static void App_Ui_Center(const char *string, uint8 half)`	按照内容宽度把中文结果或提示放在一行中央。 被引用：App_Ui_Render。
App_Ui_Big_Time 第185到207行 `static void App_Ui_Big_Time(uint8 half, const char *string)`	将 5x7 数字字形放大为时钟使用的双页字形。 被引用：App_Ui_Render。
App_Ui_Render 第209到334行 `static void App_Ui_Render(void)`	根据角色、网络、RTC、数据库和模式状态生成当前 128 字节页面。 被引用：App_Ui_Tick。
App_Ui_Init 第336到342行 `void App_Ui_Init(uint8 terminal)`	初始化 OLED 并保存协调器或终端的界面角色。 被引用：App_Coordinator_Peripherals_Process、App_Terminal_Peripherals_Process。
App_Ui_Status 第344到358行 `void App_Ui_Status(uint8 joined, uint8 peer, uint8 database, uint8 count, uint8 new_mode, uint8 remaining_seconds)`	保存网络与考勤状态；状态变化时标记整屏需要重绘。 被引用：App_Coordinator_Display_Process、App_Terminal_Display_Process。
App_Ui_Time 第360到365行 `void App_Ui_Time(const rtc_time_t *time, uint8 status)`	更新 RTC 显示数据；时间无效时提示时钟未校时。 被引用：App_Coordinator_Peripherals_Process、App_Coordinator_Second_Process、App_Terminal_Peripherals_Process等。
App_Ui_Message 第367到373行 `void App_Ui_Message(const char *heading, const char *explanation)`	将业务提示翻译为中文并显示五秒，学号仍显示原始数字。 被引用：App_Coordinator_Card_Process、App_Coordinator_Set_Mode、App_Coordinator_Show_Result等。

函数及定位	用途及引用关系
App_Ui_Second 第375到383行 `void App_Ui_Second(void)`	每秒递减结果提示寿命，结束时恢复主界面。 被引用：App_Coordinator_Second_Process、App_Terminal_Second_Process。
App_Ui_Error 第385到388行 `uint8 App_Ui_Error(void)`	返回最近一次 OLED 初始化或分段写入错误。 被引用：App_Coordinator_Network_Status、App_Coordinator_Peripherals_Process、App_Terminal_Network_Status等。
App_Ui_Tick 第391到427行 `void App_Ui_Tick(void)`	每次 OSAL 事件最多发送当前页的 31 字节，避免整屏传输阻塞无线任务。 被引用：App_Coordinator_Display_Process、App_Terminal_Display_Process。

UART缓冲模块

文件：hal/hal_console_uart.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Uart_Rx_Callback 第23到28行 `static void Hal_Uart_Rx_Callback(uint8 port, uint8 event)`	UART接收回调只置应用RX事件，完整命令在任务中解析。 被引用：Hal_Uart_Init。
Hal_Uart_Init 第29到49行 `void Hal_Uart_Init(uint8 task, uint16 event)`	独占打开 UART0，配置 38400 波特率并清空收发状态。 被引用：SampleApp_Init。
Hal_Uart_Queue_Byte 第50到64行 `static void Hal_Uart_Queue_Byte(char c)`	单字节入队，满时增加丢字统计，禁止在OSAL中阻塞等待。 被引用：Hal_Uart_Print、Hal_Uart_Printf、Hal_Uart_Write。
Hal_Uart_Write 第66到78行 `void Hal_Uart_Write(const uint8 *data, uint8 length)`	按明确字节长度将旧串口发送数据放入唯一TX缓存，不等待硬件。 被引用：Device1_UART_SendData、Device1_UART_SendString。
Hal_Uart_Print 第80到86行 `void Hal_Uart_Print(char *text)`	把以零结尾的字符串排入异步发送缓冲。 被引用：App_Coordinator_Begin_Mode、App_Coordinator_Command_Process、App_Coordinator_Console_Process等。
Hal_Uart_Printf 第87到141行 `void Hal_Uart_Printf(char *format, ...)`	轻量格式化输出，仅支持本项目使用的 s、c、x、u 格式。 被引用：App_Coordinator_Card_Process、App_Coordinator_Command_Process、App_Coordinator_Network_Status等。
Hal_Uart_Flush 第142到159行 `void Hal_Uart_Flush(void)`	每次发送至多 32 字节，按 HAL 实际接受字节数推进队头。 被引用：SampleApp_ProcessEvent。

函数及定位	用途及引用关系
Hal_Uart_Set_Handler 第161到164行 `void Hal_Uart_Set_Handler(void (*handler)(char *))`	注册接收完整命令行后的同步业务处理函数。 被引用：SampleApp_Init。
Hal_Uart_Free 第165到168行 `uint8 Hal_Uart_Free(void)`	返回当前发送缓冲区可用字节数。 被引用：App_Coordinator_Command_Process、App_Coordinator_Console_Process、App_Terminal_Command_Process等。
Hal_Uart_Process_Rx 第169到224行 `uint8 Hal_Uart_Process_Rx(void)`	每次读取至多 32 字节，处理 CR/LF、退格、超长输入并再次置事件继续排空。 被引用：SampleApp_ProcessEvent。

按键消抖模块

文件：hal/hal_board_key.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Key_Init 第10到13行 `void Hal_Key_Init(void)`	只重置项目消抖状态，GPIO初始化由SDK负责。 被引用：App_Coordinator_Init、App_Terminal_Init。
Hal_Key_Poll 第14到36行 `uint8 Hal_Key_Poll(void)`	每10ms采样SDK键值，3次稳定后只返回新按下边沿，长按不重复。 被引用：SampleApp_ProcessEvent。

独立蜂鸣器模块

文件：hal/hal_buzzer.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Buzzer_Init 第9到15行 `void Hal_Buzzer_Init(void)`	先写关闭电平再切GPIO输出，避免初始化脉冲。 被引用：App_Coordinator_Init、App_Terminal_Init。
Hal_Buzzer_Bind 第16到20行 `void Hal_Buzzer_Bind(uint8 task, uint16 event)`	登记蜂鸣器关闭事件的应用任务ID和事件位。 被引用：App_Coordinator_Init、App_Terminal_Init。
Hal_Buzzer_On 第21到24行 `void Hal_Buzzer_On(void)`	P0.0高电平打开蜂鸣器，不负责业务持续时间。 被引用：Hal_Buzzer_Beep。
Hal_Buzzer_Off 第25到28行 `void Hal_Buzzer_Off(void)`	P0.0低电平关闭蜂鸣器，供定时事件和失败清理调用。 被引用：Hal_Buzzer_Beep、SampleApp_ProcessEvent。
Hal_Buzzer_Beep 第29到43行 `void Hal_Buzzer_Beep(uint16 ms)`	关闭旧输出、停止旧计时，成功安排关闭事件后才响，资源失败保持静音。 被引用：App_Coordinator_Command_Process、App_Coordinator_Show_Result、App_Terminal_Command_Process等。

软件I2C模块

文件：hal/hal_i2c.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_I2c_Delay 第11到14行 `static void Hal_I2c_Delay(void)`	提供软件 I2C 半周期等待。 被引用：Hal_I2c_Init、Hal_I2c_Read_Byte、Hal_I2c_Scl_Release等。
Hal_I2c_Scl_Release 第15到30行 `static uint8 Hal_I2c_Scl_Release(void)`	释放开漏时钟线并等待外设时钟拉伸结束。 被引用：Hal_I2c_Init、Hal_I2c_Read_Byte、Hal_I2c_Start等。
Hal_I2c_Stop 第31到45行 `uint8 Hal_I2c_Stop(void)`	产生停止条件并释放数据线。 被引用：Hal_I2c_Finish、Hal_I2c_Init。
Hal_I2c_Init 第46到72行 `uint8 Hal_I2c_Init(void)`	释放总线、尝试九个恢复时钟并检查 SDA 是否回到高电平。 被引用：App_Coordinator_Peripherals_Process、App_Terminal_Peripherals_Process。
Hal_I2c_Start 第73到95行 `uint8 Hal_I2c_Start(void)`	产生起始或重复起始条件。 被引用：Hal_I2c_Address。
Hal_I2c_Write_Byte 第96到130行 `uint8 Hal_I2c_Write_Byte(uint8 value)`	按高位优先写八位，并读取外设 ACK/NACK。 被引用：Hal_I2c_Address、Hal_I2c_Read_Reg、Hal_I2c_Write等。
Hal_I2c_Read_Byte 第131到170行 `uint8 Hal_I2c_Read_Byte(uint8 *value, uint8 ack)`	读八位后按 ack 参数回应 ACK 或 NACK。 被引用：Hal_I2c_Read、Hal_I2c_Read_Reg。
Hal_I2c_Finish 第171到175行 `static uint8 Hal_I2c_Finish(uint8 status)`	无论前一步是否失败都尝试发送 STOP，并保留先发生的错误。 被引用：Hal_I2c_Probe、Hal_I2c_Read、Hal_I2c_Read_Reg等。
Hal_I2c_Address 第176到180行 `static uint8 Hal_I2c_Address(uint8 addr, uint8 read)`	发送七位从地址与读写方向位。 被引用：Hal_I2c_Probe、Hal_I2c_Read、Hal_I2c_Read_Reg等。
Hal_I2c_Probe 第181到190行 `uint8 Hal_I2c_Probe(uint8 addr)`	只发送地址检测设备应答。 被引用：Hal_Oled_Init。
Hal_I2c_Write 第191到204行 `uint8 Hal_I2c_Write(uint8 addr, const uint8 *data, uint8 len)`	向设备写入最多 32 字节原始数据。 被引用：Hal_Eeprom_Init、Hal_Eeprom_Process、Hal_Oled_Clear等。
Hal_I2c_Read 第205到218行 `uint8 Hal_I2c_Read(uint8 addr, uint8 *data, uint8 len)`	从设备读取最多 32 字节原始数据。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。

函数及定位	用途及引用关系
Hal_I2c_Write_Reg 第219到236行 `uint8 Hal_I2c_Write_Reg(uint8 addr, uint8 reg, const uint8 *data, uint8 len)`	先发送设备内寄存器地址，再写入数据。 被引用：Hal_Eeprom_Process、Hal_Rtc_Set_Time。
Hal_I2c_Read_Reg 第237到258行 `uint8 Hal_I2c_Read_Reg(uint8 addr, uint8 reg, uint8 *data, uint8 len)`	先写寄存器地址，再重复起始读取数据。 被引用：Hal_Eeprom_Read、Hal_Rtc_Init、Hal_Rtc_Read_Time。

EEPROM驱动模块

文件：hal/hal_eeprom.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Eeprom_Device 第11到14行 `static uint8 Hal_Eeprom_Device(uint16 addr)`	按 AT24C08 地址高位选择 256 字节块对应的 I2C 从地址。 被引用：Hal_Eeprom_Process、Hal_Eeprom_Read。
Hal_Eeprom_Configure 第15到28行 `uint8 Hal_Eeprom_Configure(uint16 size, uint8 page)`	在 AT24C02 与 AT24C08 两种容量/页大小之间切换。 被引用：App_Database_Init、App_Host_Store_Init。
Hal_Eeprom_Init 第29到32行 `uint8 Hal_Eeprom_Init(void)`	通过从地址探测 EEPROM 是否应答。 被引用：App_Coordinator_Peripherals_Process。
Hal_Eeprom_Read 第33到63行 `uint8 Hal_Eeprom_Read(uint16 addr, uint8 *data, uint8 len)`	跨 256 字节块边界分段读取，单次 I2C 传输最多 32 字节。 被引用：App_Database_Tick、App_Host_Store_Process、Hal_Eeprom_Read_Byte。
Hal_Eeprom_Read_Byte 第64到67行 `uint8 Hal_Eeprom_Read_Byte(uint16 addr, uint8 *value)`	读取指定地址的单字节。 被引用：App_Database_Tick、App_Host_Store_Process。
Hal_Eeprom_Write 第68到85行 `uint8 Hal_Eeprom_Write(uint16 addr, const uint8 *data, uint8 len)`	复制写入数据到私有缓存并启动异步分页写流程。 被引用：App_Database_Tick、App_Host_Store_Process、Hal_Eeprom_Write_Byte。
Hal_Eeprom_Write_Byte 第86到89行 `uint8 Hal_Eeprom_Write_Byte(uint16 addr, uint8 value)`	以同样的异步写流程写入单字节。 被引用：App_Database_Tick、App_Host_Store_Process。
Hal_Eeprom_Is_Busy 第90到93行 `uint8 Hal_Eeprom_Is_Busy(void)`	报告写入状态，供上层避免并发访问。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。
Hal_Eeprom_Process 第94到141行 `uint8 Hal_Eeprom_Process(void)`	每次推进一页写入或 ACK 轮询；超时后返回最终错误。 被引用：App_Database_Tick、App_Host_Store_Process。

RTC驱动模块

文件：hal/hal_rtc.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Rtc_Bcd 第13到16行 `static uint8 Hal_Rtc_Bcd(uint8 n)`	把十进制日期时间字段编码为 RTC 使用的 BCD。 被引用：Hal_Rtc_Set_Time。
Hal_Rtc_Decimal 第17到24行 `static uint8 Hal_Rtc_Decimal(uint8 n)`	把 BCD 字段解码为十进制，非法 BCD 返回 0xFF。 被引用：Hal_Rtc_Read_Time。
Hal_Rtc_Valid 第25到43行 `static uint8 Hal_Rtc_Valid(const rtc_time_t *t)`	校验年月日、闰年、星期与时分秒范围。 被引用：Hal_Rtc_Read_Time、Hal_Rtc_Set_Time。
Hal_Rtc_Init 第44到51行 `uint8 Hal_Rtc_Init(uint8 *flags)`	读取 RTC 状态标志；不在初始化时擅自改写时间。 被引用：App_Coordinator_Peripherals_Process、App_Terminal_Peripherals_Process、Hal_Rtc_Read_Time。
Hal_Rtc_Read_Time 第52到105行 `uint8 Hal_Rtc_Read_Time(rtc_time_t *time)`	检测掉电/停振标志，读取并校验日期时间；无效则返回专用状态码。 被引用：App_Coordinator_Card_Process、App_Coordinator_Command_Process、App_Coordinator_Peripherals_Process等。
Hal_Rtc_Set_Time 第106到141行 `uint8 Hal_Rtc_Set_Time(const rtc_time_t *t)`	先停止计时、写入完整日期与扩展寄存器，再释放停止位运行 RTC。 被引用：App_Coordinator_Set_Time、App_Terminal_Set_Time、Hal_Rtc_Seed_Build_Time。
Hal_Rtc_Seed_Build_Time 第143到198行 `uint8 Hal_Rtc_Seed_Build_Time(rtc_time_t *time)`	RTC 时间无效时，以本地编译日期时间设定时钟并回读验证。 被引用：App_Coordinator_Peripherals_Process。

RFID驱动模块

文件：hal/hal_rc522.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Rc522_Spi_Byte 第52到66行 `static uint8 Hal_Rc522_Spi_Byte(uint8 value)`	软件 SPI 按高位优先同步收发一个字节。 被引用：Hal_Rc522_Read_Reg、Hal_Rc522_Write_Reg。
Hal_Rc522_Read_Reg 第67到75行 `uint8 Hal_Rc522_Read_Reg(uint8 reg)`	通过软件 SPI 读取 RC522 寄存器。 被引用：Hal_Rc522_Exchange、Hal_Rc522_Init、Hal_Rc522_Reset。
Hal_Rc522_Write_Reg 第76到82行 `void Hal_Rc522_Write_Reg(uint8 reg, uint8 value)`	通过软件 SPI 写入 RC522 寄存器。 被引用：Hal_Rc522_Exchange、Hal_Rc522_Init、Hal_Rc522_Reset。
Hal_Rc522_Fail 第83到87行 `static bool Hal_Rc522_Fail(uint8 error)`	保存最近一次 RC522 错误并返回失败。 被引用：Hal_Rc522_Anticoll、Hal_Rc522_Exchange、Hal_Rc522_Init等。

函数及定位	用途及引用关系
Hal_Rc522_Reset 第88到104行 `bool Hal_Rc522_Reset(void)`	执行芯片软复位并限时轮询完成状态。 被引用：Hal_Rc522_Init。
Hal_Rc522_Init 第105到138行 `bool Hal_Rc522_Init(void)`	配置 SPI 引脚、确认版本寄存器、设置射频定时器并打开天线。 被引用：App_Coordinator_Peripherals_Process、App_Terminal_Peripherals_Process。
Hal_Rc522_Crc_A 第139到151行 `static void Hal_Rc522_Crc_A(const uint8 *data, uint8 len, uint8 *result)`	计算 ISO14443A CRC_A，供 SELECT 和 HALT 命令使用。 被引用：Hal_Rc522_Read_Uid、Hal_Rc522_Select。
Hal_Rc522_Exchange 第152到200行 `static bool Hal_Rc522_Exchange(const uint8 *send, uint8 send_len, uint8 bits, uint8 *recv, uint8 expected)`	清 FIFO、发送命令、等待 IRQ 并检查接收字节数与协议错误。 被引用：Hal_Rc522_Anticoll、Hal_Rc522_Read_Uid、Hal_Rc522_Request等。
Hal_Rc522_Select_Command 第201到204行 `static uint8 Hal_Rc522_Select_Command(uint8 level)`	按级联层级选择 CL1、CL2 或 CL3 的 SELECT 命令字。 被引用：Hal_Rc522_Anticoll、Hal_Rc522_Select。
Hal_Rc522_Request 第205到213行 `bool Hal_Rc522_Request(uint8 atqa[2])`	发送 WUPA 唤醒卡片并取得 ATQA。 被引用：Hal_Rc522_Read_Uid。
Hal_Rc522_Anticoll 第214到232行 `bool Hal_Rc522_Anticoll(uint8 level, uint8 block[5])`	执行当前级联层的防冲突过程并校验 BCC。 被引用：Hal_Rc522_Read_Uid。
Hal_Rc522_Select 第233到255行 `bool Hal_Rc522_Select(uint8 level, const uint8 block[5], uint8 *sak)`	选择当前级联层并核对 SAK 回复的 CRC_A。 被引用：Hal_Rc522_Read_Uid。
Hal_Rc522_Read_Uid 第256到296行 `bool Hal_Rc522_Read_Uid(uint8 uid[RC522_UID_MAX], uint8 *len)`	逐级联读取 4、7 或 10 字节 UID；结束时发送 HALT，允许下次重新唤醒。 被引用：App_Coordinator_Card_Process、App_Terminal_Card_Process。

OLED硬件驱动模块

文件：hal/hal_oled.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Oled_Set_Position 第24到32行 `static uint8 Hal_Oled_Set_Position(uint8 page, uint8 column)`	向 SSD1306 发送页地址和列地址命令。 被引用：Hal_Oled_Clear、Hal_Oled_Clear_Page、Hal_Oled_Show_Line等。
Hal_Oled_Clear 第34到65行 `static uint8 Hal_Oled_Clear(void)`	按页清空整屏，单次 I2C 写入不超过 31 个显示字节。 被引用：Hal_Oled_Init。
Hal_Oled_Clear_Page 第67到95行 `static uint8 Hal_Oled_Clear_Page(uint8 page)`	只清空指定页，供文本行更新。 被引用：Hal_Oled_Show_Line。

函数及定位	用途及引用关系
Hal_Oled_Glyph 第97到138行 `void Hal_Oled_Glyph(uint8 c, uint8 glyph[6])`	将数字、英文字母和常用标点转换为六列字形。 被引用：App_Ui_Big_Time、App_Ui_Text、Hal_Oled_Show_Line。
Hal_Oled_Init 第140到194行 `uint8 Hal_Oled_Init(void)`	依次探测 0x3C/0x3D 地址，配置 SSD1306 并清屏。 被引用：App_Ui_Init。
Hal_Oled_Get_Address 第196到199行 `uint8 Hal_Oled_Get_Address(void)`	返回探测成功的 OLED 七位 I2C 地址。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。
Hal_Oled_Show_Line 第201到246行 `uint8 Hal_Oled_Show_Line(uint8 page, const char *text)`	清除指定页后写入最多 21 个 ASCII 字符。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。
Hal_Oled_Write_Chunk 第249到267行 `uint8 Hal_Oled_Write_Chunk(uint8 page, uint8 x, const uint8 *data, uint8 len)`	从指定页和列写入不超过 31 字节的像素分片。 被引用：App_Ui_Tick。

保留的延时接口

文件：hal/hal_delay.c。按以下源码顺序逐个函数阅读。

函数及定位	用途及引用关系
Hal_Delay_Us 第3到18行 `void Hal_Delay_Us(uint16 us)`	保留原NOP循环忙等接口，未用仪器校准，不用于长业务等待。 被引用：Hal_Delay_Ms。
Hal_Delay_Ms 第20到26行 `void Hal_Delay_Ms(uint16 ms)`	重复调用短忙等接口，当前主业务没有以此替代OSAL定时器。 当前自写主业务无引用，属于保留接口；测试或外部SDK引用需另查。

SDK继续阅读入口

模块	从哪个入口继续
ZMain.c	main，硬件、AF和OSAL系统初始化
OSAL_SampleApp.c	tasksArr、osalInitTasks，注册顺序
OSAL.c及OSAL_Timers.c	osal_run_system、事件位、消息和定时器
ZDApp.c及ZDO模块	ZDApp_Init、ZDOInitDevice、ZDO_NETWORK_INIT、ZDO_StartDevice
AF.c	afRegister、AF_DataRequest，端点及发送分派
hal_board_cfg.h	GAOCHANG_RC522_BOARD分支，LED及按键适配
hal_led.c及hal_key.c	HalLedSet、HalKeyRead，直接SDK驱动
hal_uart.c及_hal_uart_isr.c	HalUARTOpen、HalUARTRead、HalUARTWrite
f8wConfig.cfg及角色cfg	PAN、信道、安全、轮询与角色编译宏
f8w2530.xcl与SampleApp.map	CODE银行、XDATA布局及实际占用

源码和核查资料

本册源码入口为E:/MCU/zigbee/RC522大作业。全部定位和函数数从当前源码提取，源文件哈希和调用索引保存在同目录源码索引.json。行号对应2026年10月13日核查快照，后续修改后应以函数名重新定位。

项目内部主要资料为SampleApp.ewp、Source/OSAL_SampleApp.c、Source/Coordinator.c、Source/EndDevice.c、app及hal的在用C与头文件，以及SDK的ZMain、OSAL、ZDApp、AF、hal_led、hal_key、hal_board_cfg及f8w配置。库、链接脚本、NV机制属于SDK，不是AT24C08考勤数据库。

TI参考：CC2530产品资料；旧版Z-Stack示例应用说明。新一代CC13xx或CC26xx的TI-RTOS、ICall应用架构不能直接套到本工程的CC2530 OSAL。